
Computer Networks
Lecture 7
Shmulik Kaplan

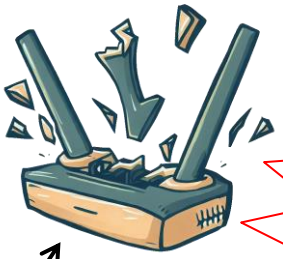
Today

- Transport layer
  - Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Principles of Reliable data transfer

The path of the packet is a **dangerous** one…





Malformed equipment



Getting Lost





Full queue



Bugs

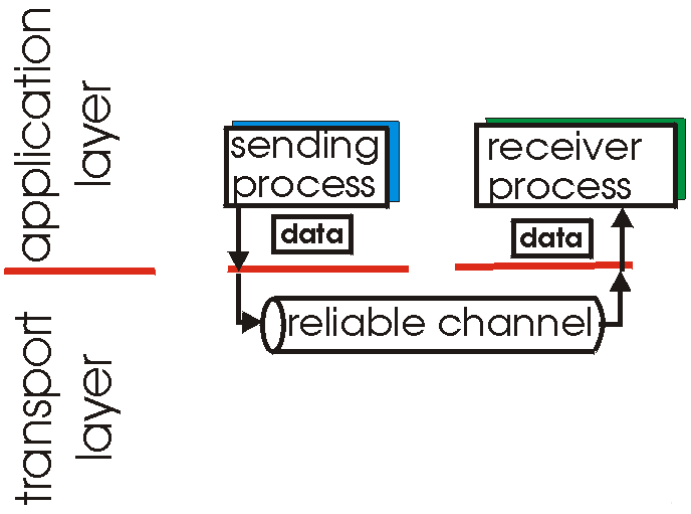


Radiation

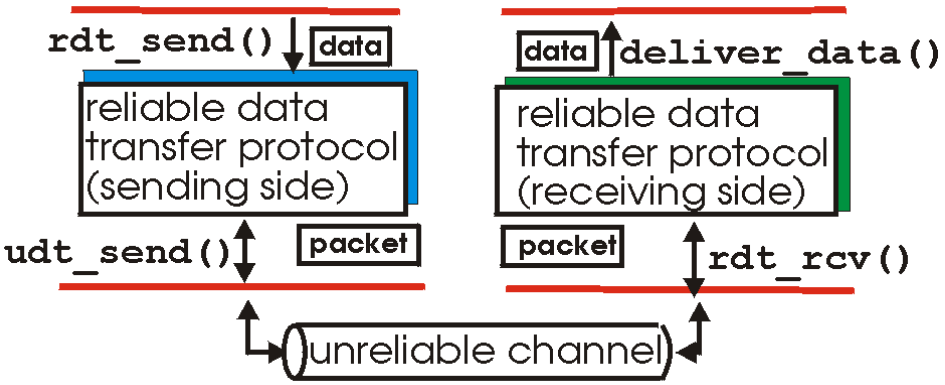
And more…

Principles of Reliable data transfer

Characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

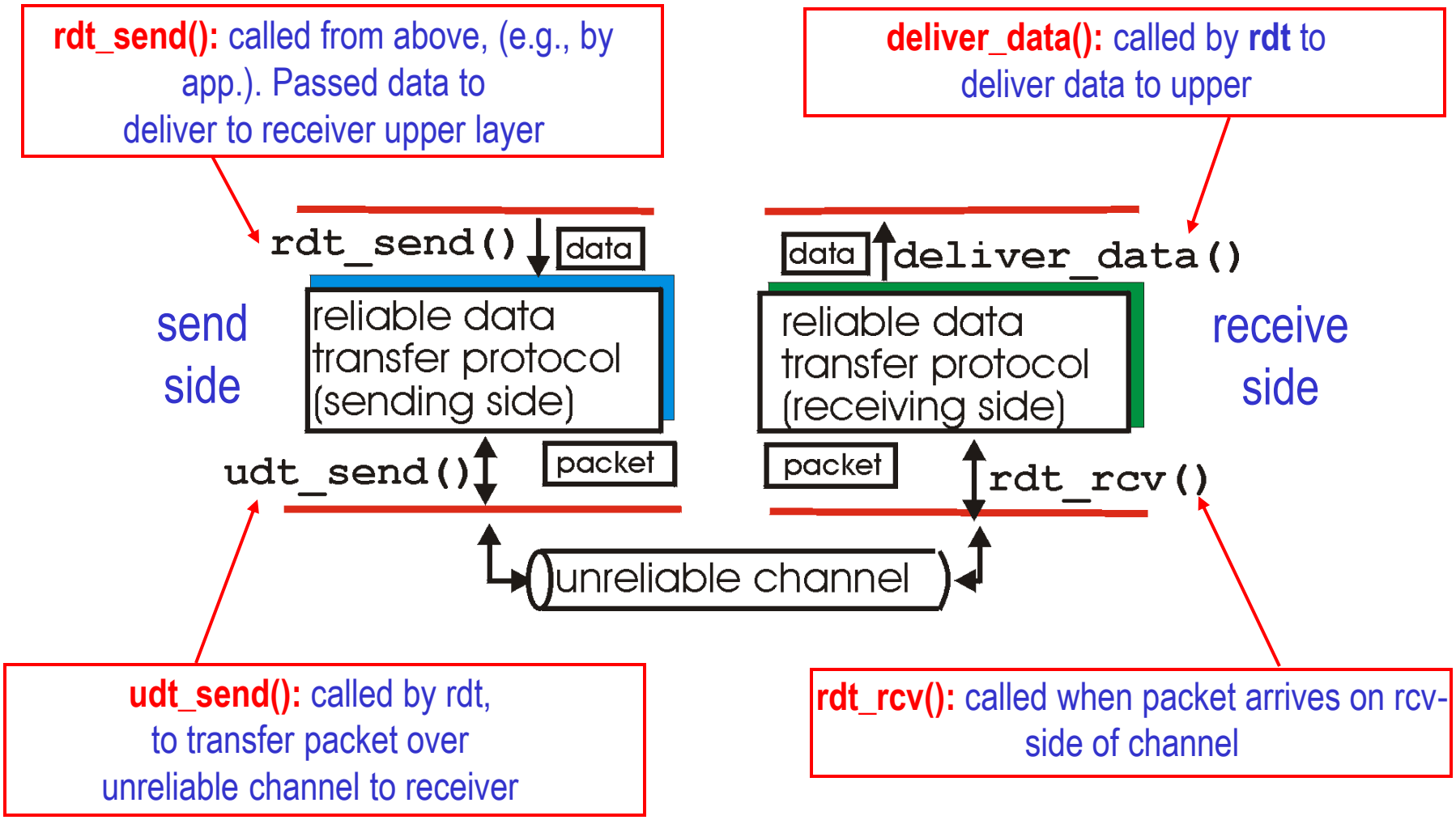


provided service



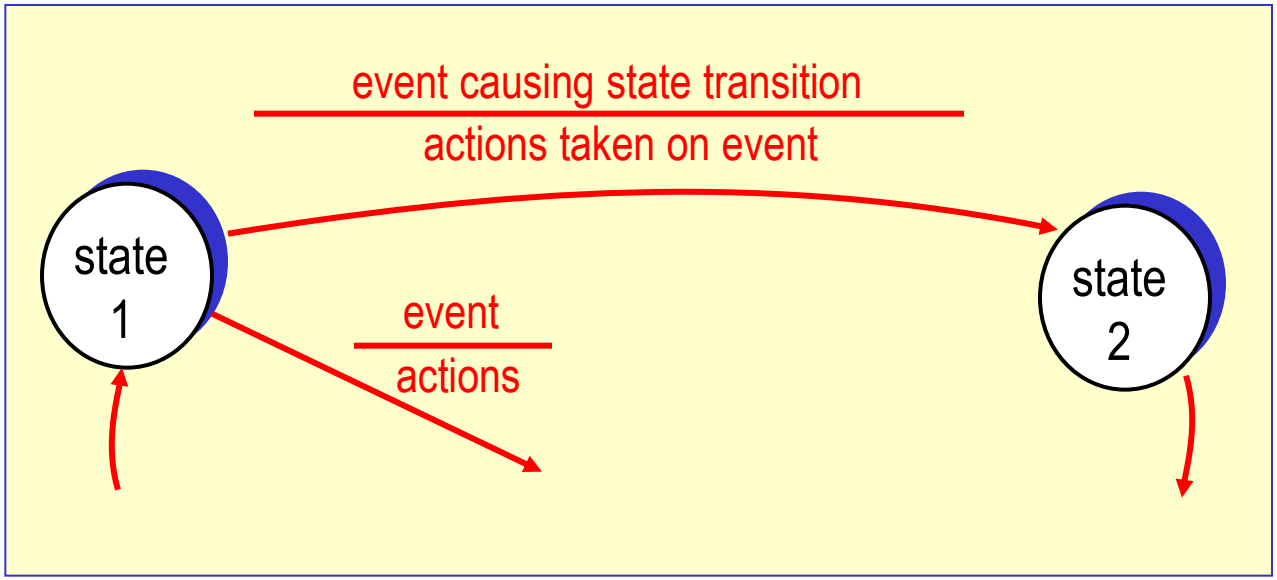
service implementation

Reliable data transfer: getting started



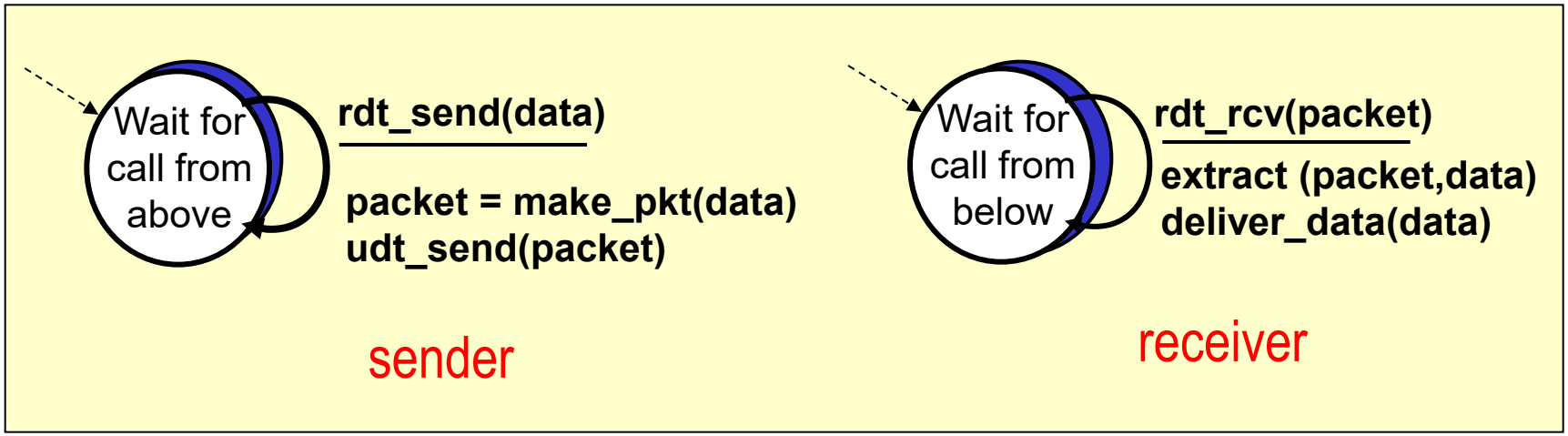
Reminder: Finite State Machine

We'll use finite state machines (FSM) to specify sender, receiver



RDT-1.0: Reliable Data Transfer Over a Reliable Channel

No Loss
No Errors
No Ordering issues



Today

- Transport layer
  - ✔ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

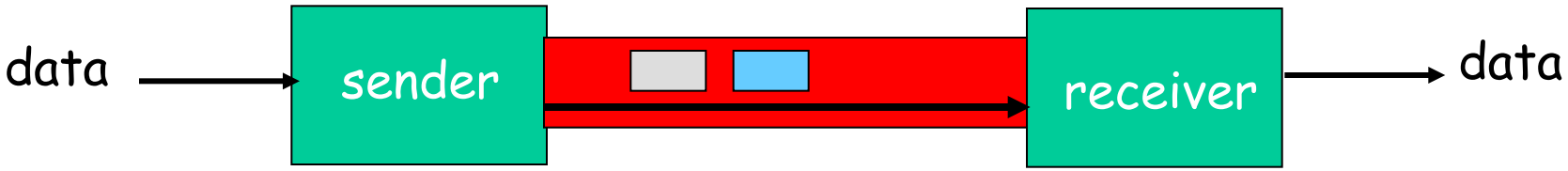
Unreliable Channel: Type of faults

1. Corrupted packets (bit errors)

2. Duplications

3. Loss

4. Reorder

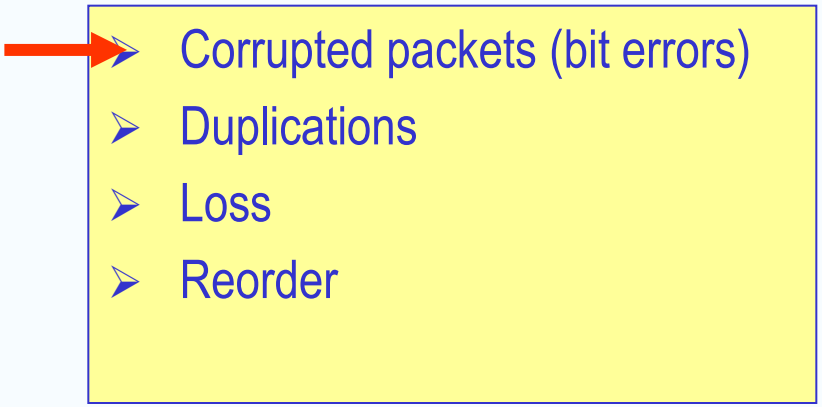


Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - Error control
    - Duplication
    - Loss
    - Reorder

RDT-2.0: Channel with bit Errors

- Underlying channel may flip bits in packet
  - can use checksum to detect bit errors
- *The* question: how to recover from errors

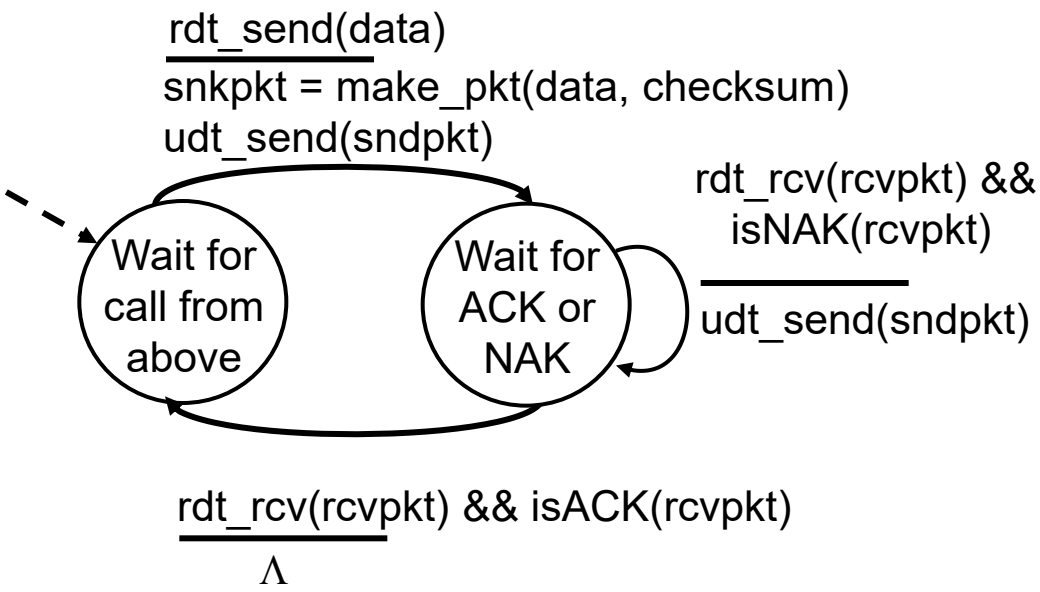


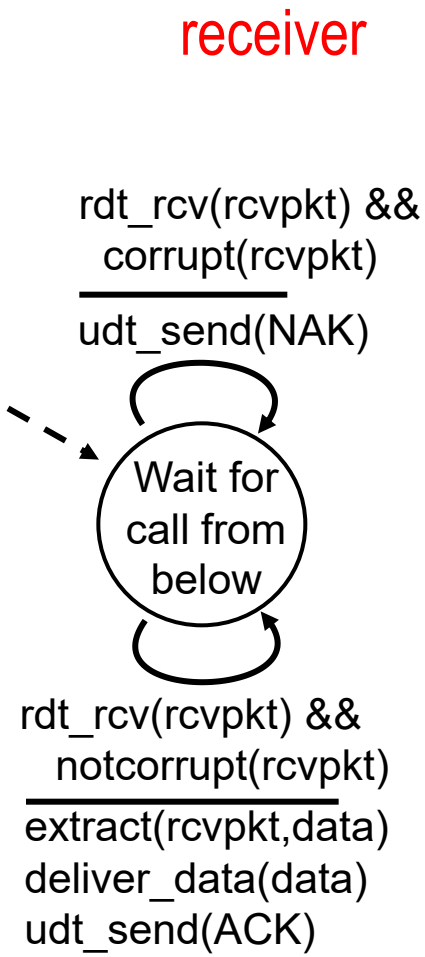
RDT-2.0: Channel with bit Errors

- Underlying channel may flip or lose bits in packet
  - can use checksum to detect bit errors
- How to recover from errors?
  - *acknowledgements (ACKs):*
    receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):*
    receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK

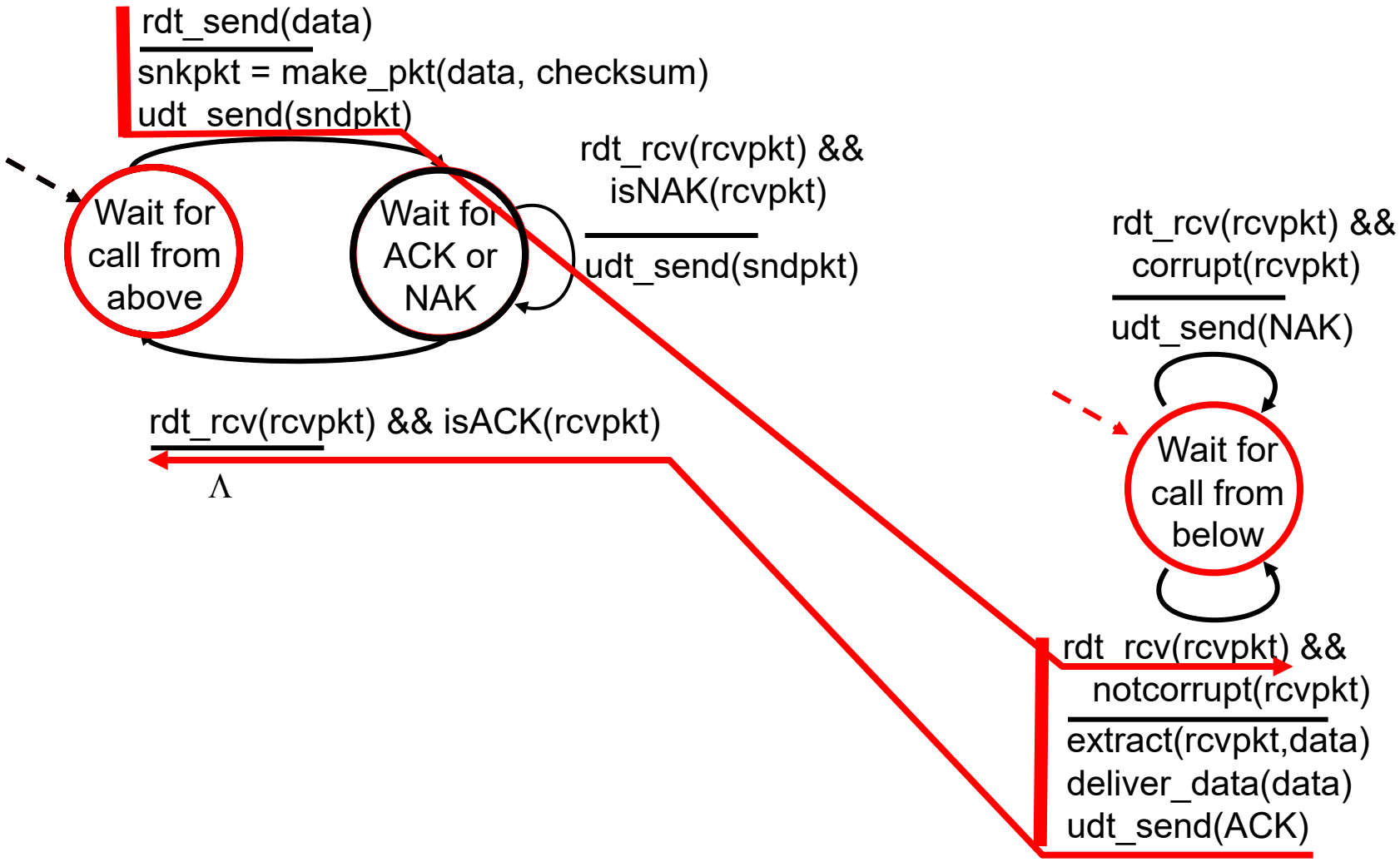
stop and wait
Sender sends one packet,
then waits for receiver response

RDT-2.0: FSM Specification

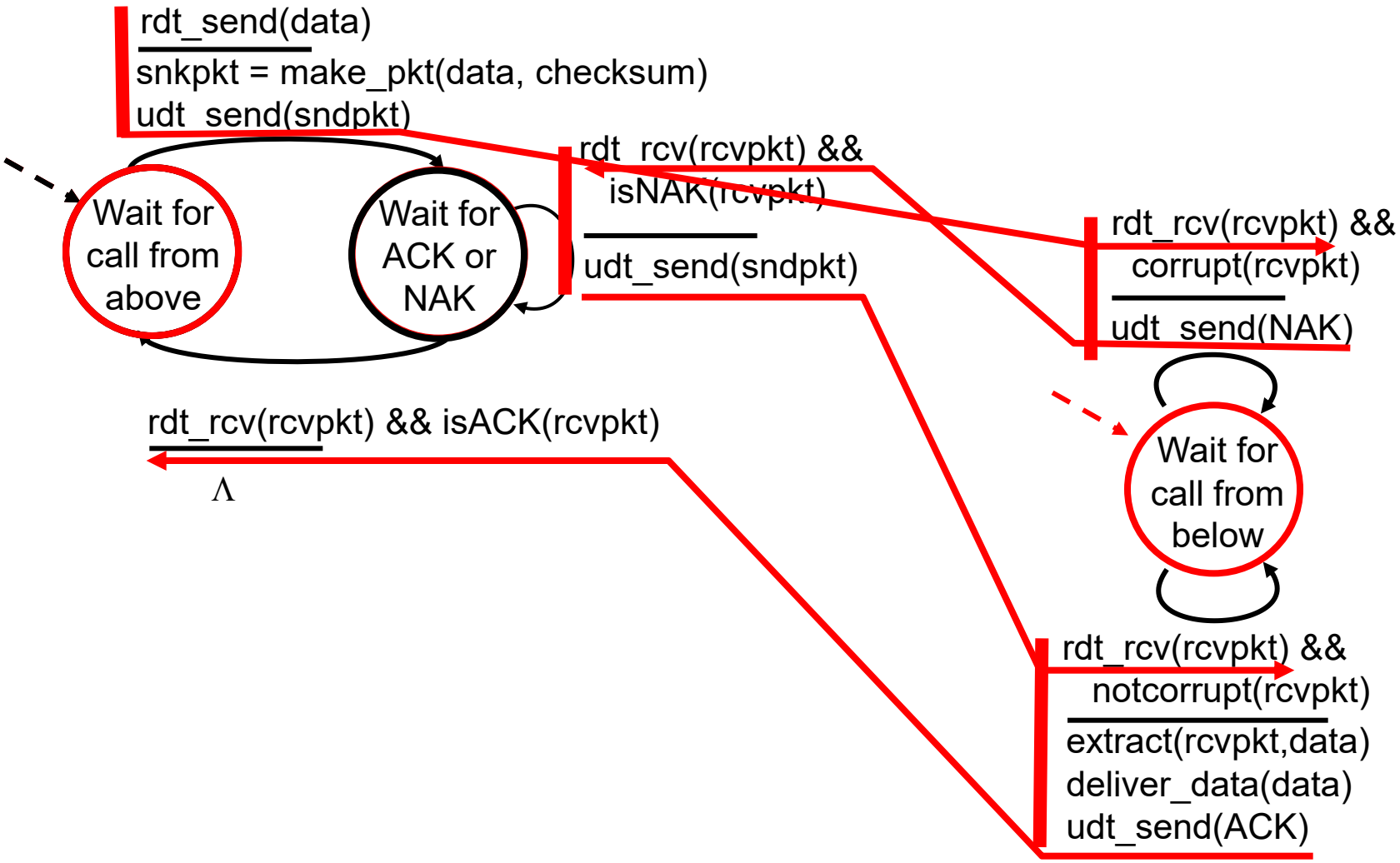




RDT-2.0: Operation with No Errors

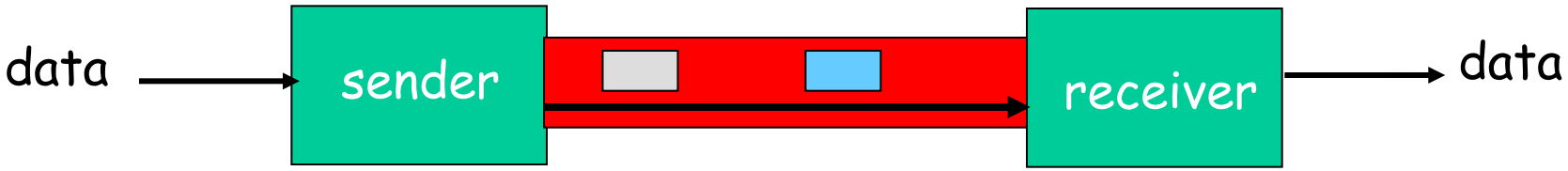


RDT-2.0: Error Scenario



RDT-2.0 A Fatal Flaw!

What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.

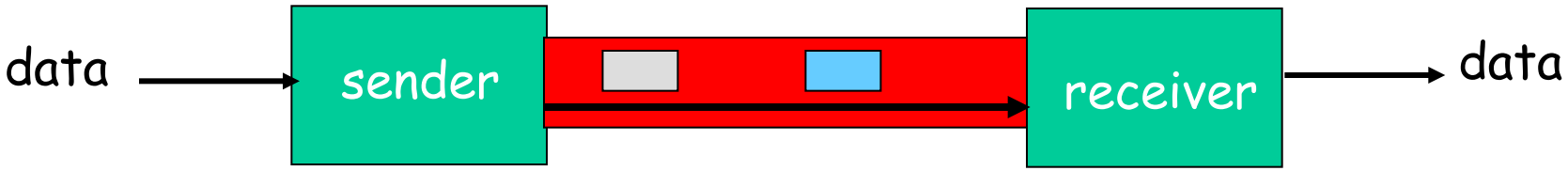


RDT-2.0 Fatal Flaw!

What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.

- sender doesn't know what happened at receiver!
- **Re-transmit**

Fatal Flow!!!



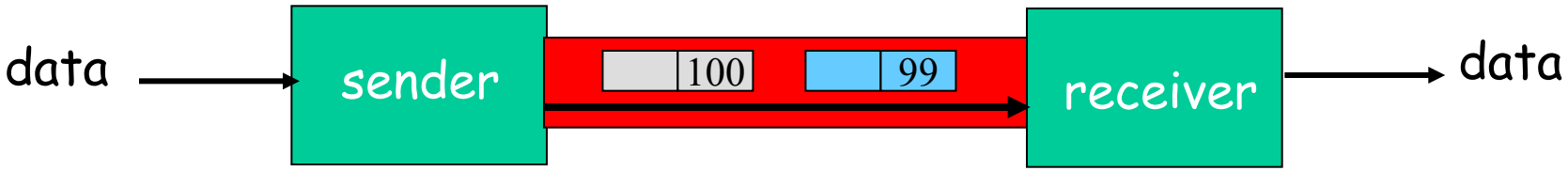
rdt2.0 has a fatal flaw!

What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.

- sender doesn't know what happened at receiver!
- **can't just retransmit -- possible duplicate**

Handling duplicates:

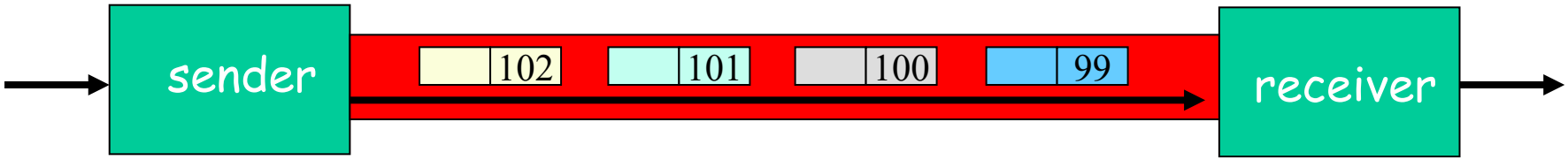
- sender adds *sequence number* to each pkt
- sender retransmits current pkt if ACK/NAK garbled
- receiver discards (doesn't deliver up) duplicate pkt



Question

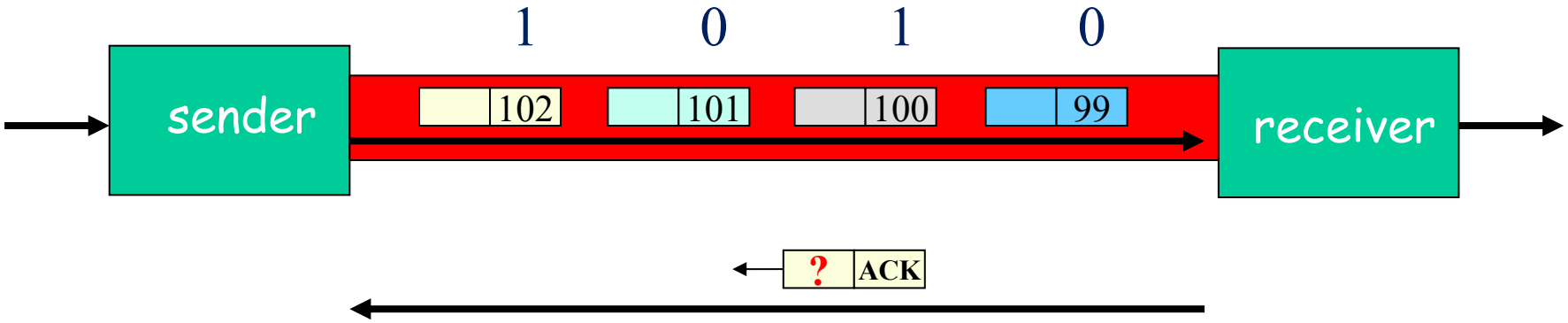
How many different sequence numbers are needed ?

And why is this of any interest?



Question

Do we need to add sequence numbers also for ACK/NAK ?



RDT-2.1: Discussion

Sender:

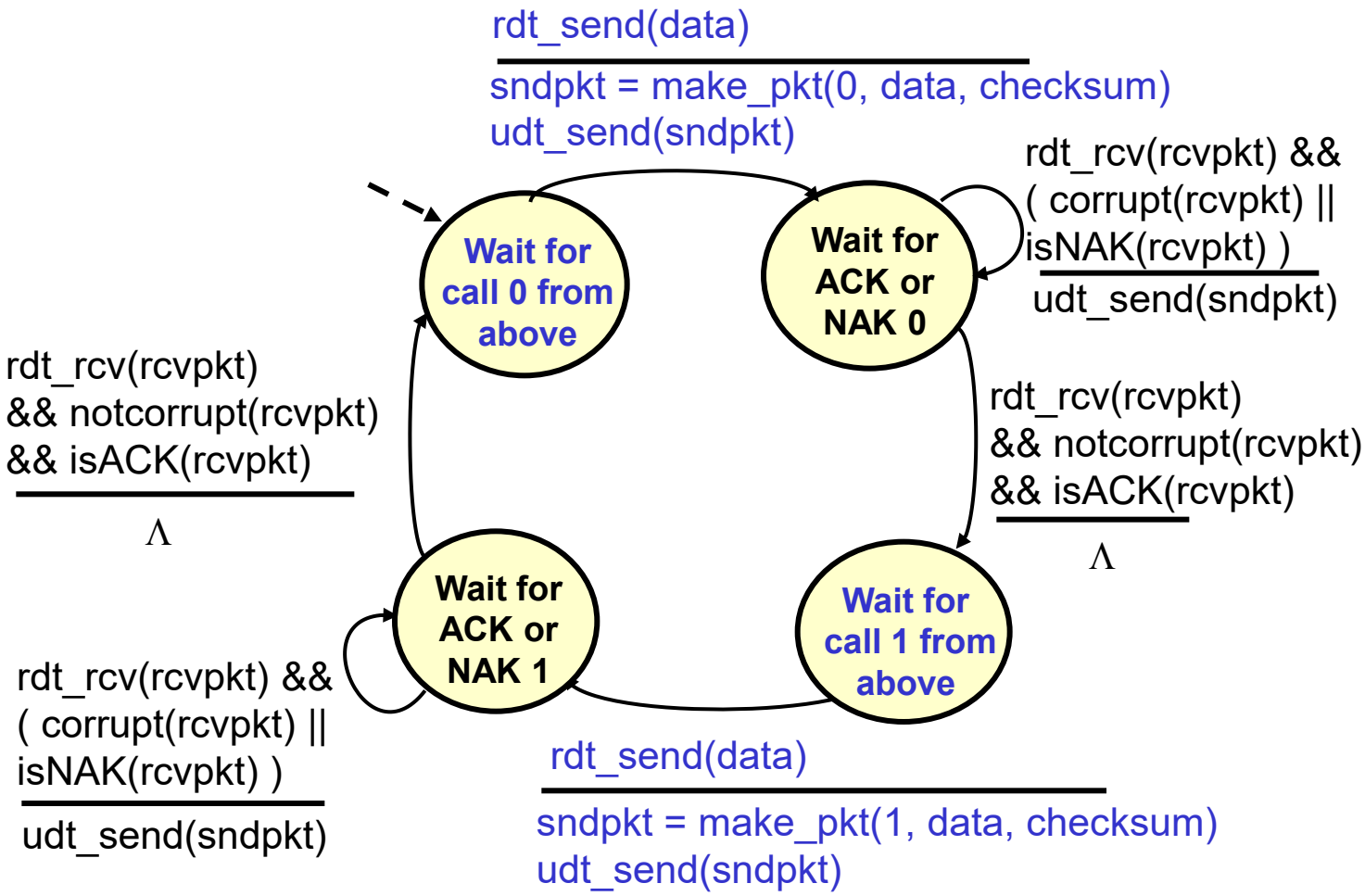
- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states

Receiver:

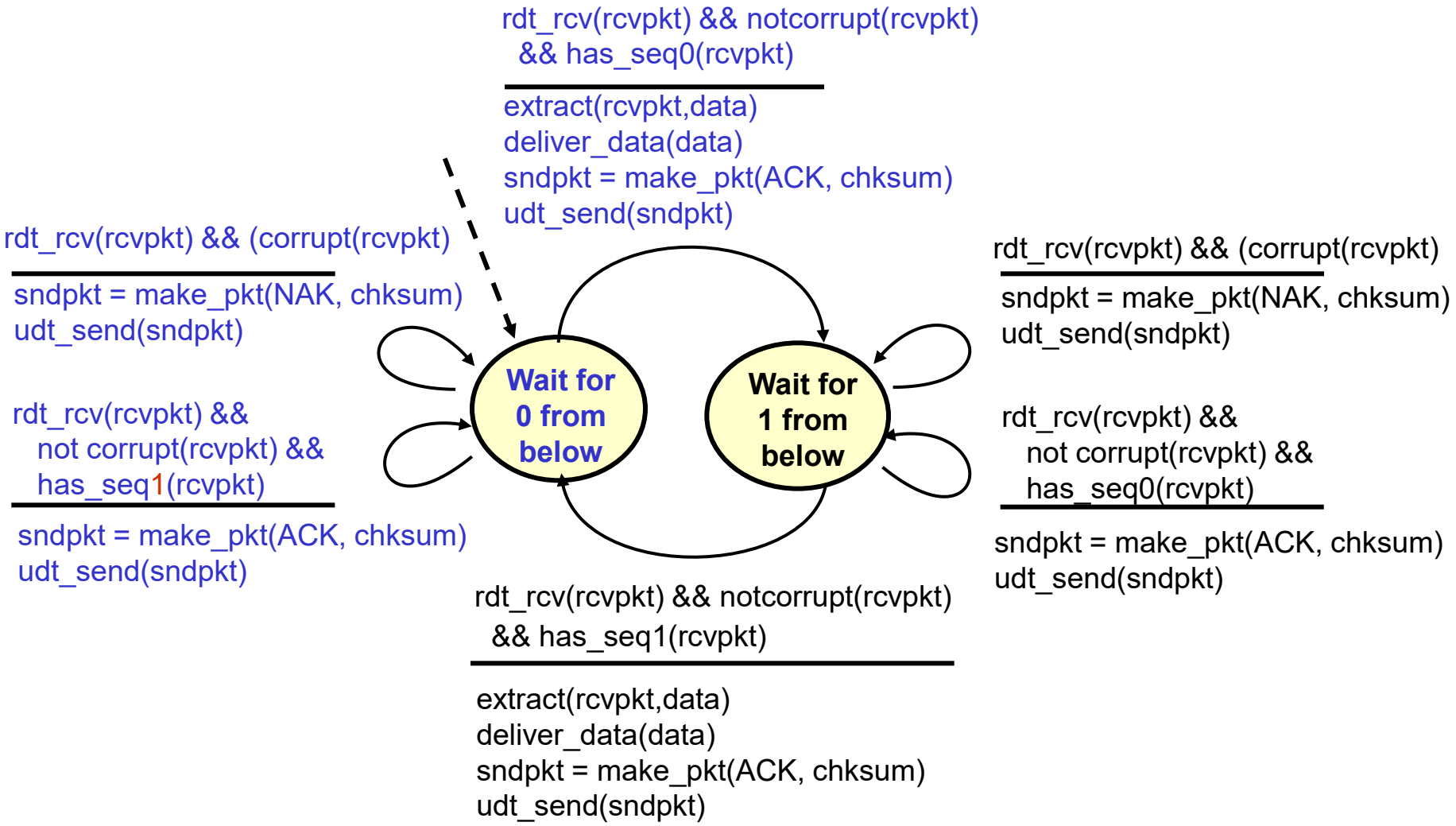
- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

The Alternating bit Protocol

RDT-2.1: Sender: Handles Garbled ACK/NAKs

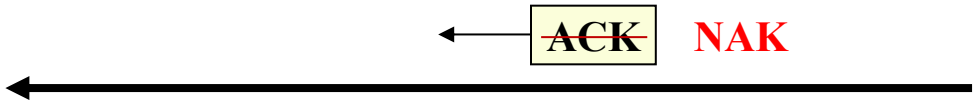


RDT-2.1: Receiver: Handles Garbled ACK/NAKs

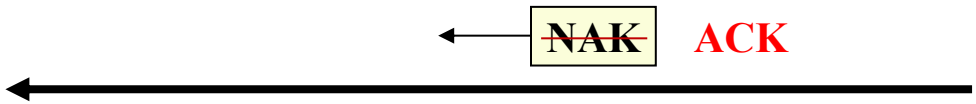


Questions

1. Would the algorithm work if ACK can be changed to NAK ?



2. Would the algorithm work if NAK can be changed to ACK ?

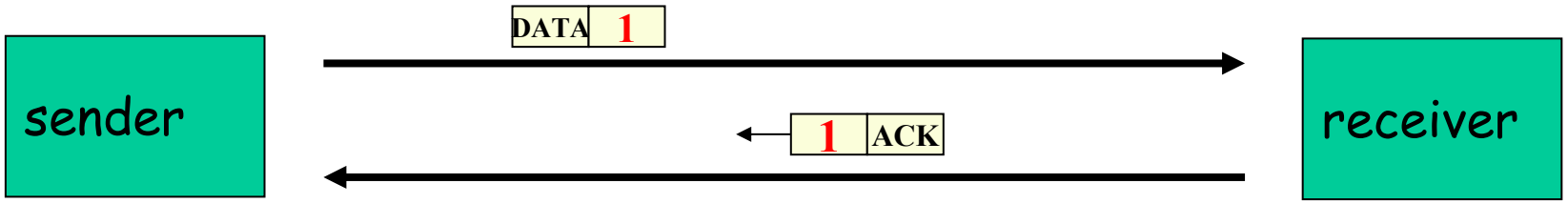


RDT-2.2: A NAK-Free Protocol

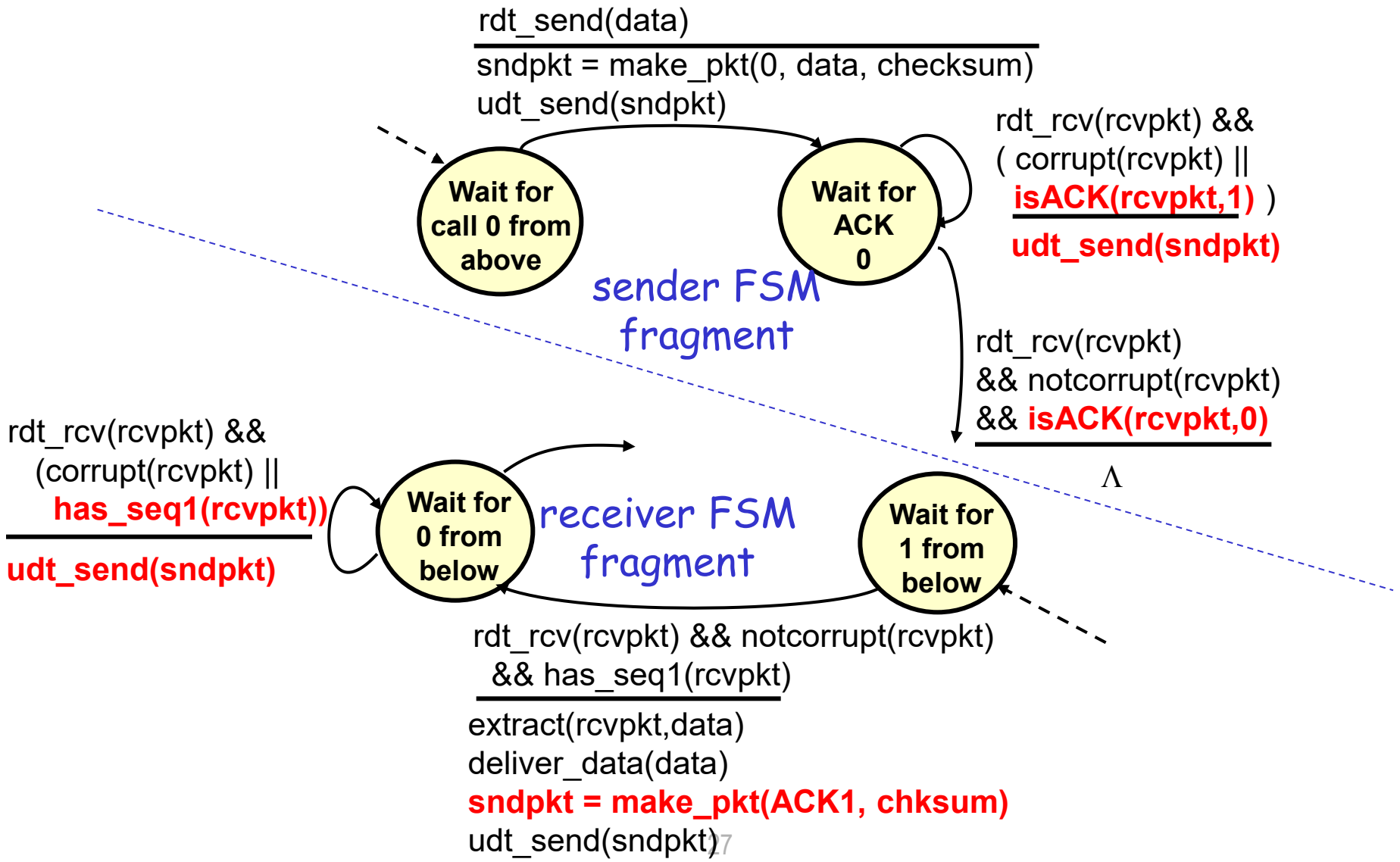
- We want the same functionality as RDT-2.1, using ACKs only

RDT-2.2: A NAK-Free Protocol

- Same functionality as rdt2.1, using ACKs only
- Instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- Duplicate ACK at sender results in same action as NAK: *retransmit current pkt*



RDT-2.2: Sender, Receiver Fragments



Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - Duplication
    - Loss
    - Reorder

Question

Would RDT-2.2 work correctly assuming duplications?

- Sender:
  - Ignores ACK already received due to packet number.
  - Receiver always sends ACK on the last received packet.
- Receiver:
  - Ignores Packet resent due to packet number
  - Sender always send forward, after ACK was received on current packet – New packet number must be the next packet

YES!!

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - Loss
    - Reorder

What about losses?

Approach: sender waits "reasonable" amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission can be duplicate, but use of seq. #'s already handles this
  - receiver must specify seq # of pkt being ACKed
- requires countdown timer

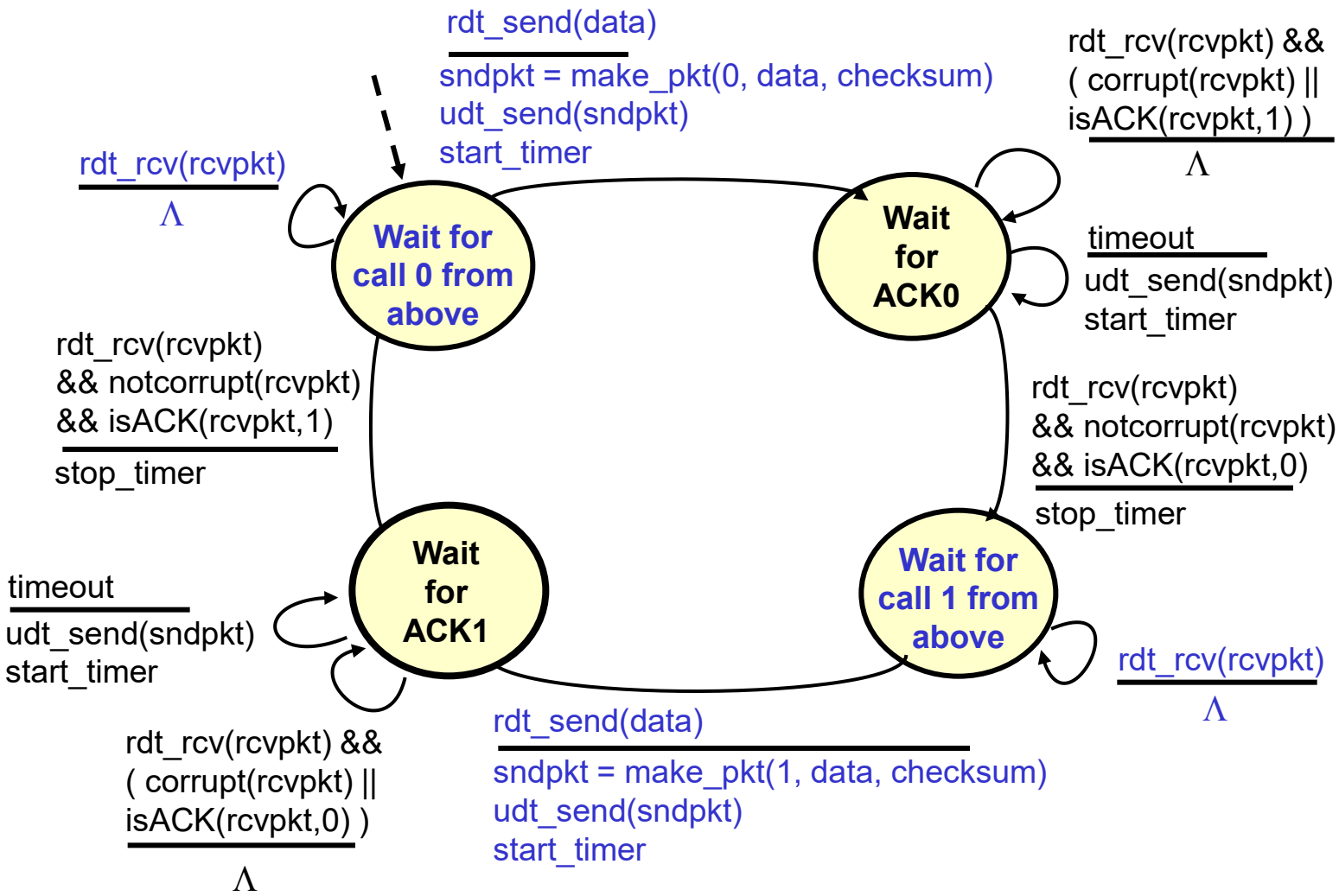
Question: A timer is required at:

1. sender only
2. receiver only
3. both sender & receiver

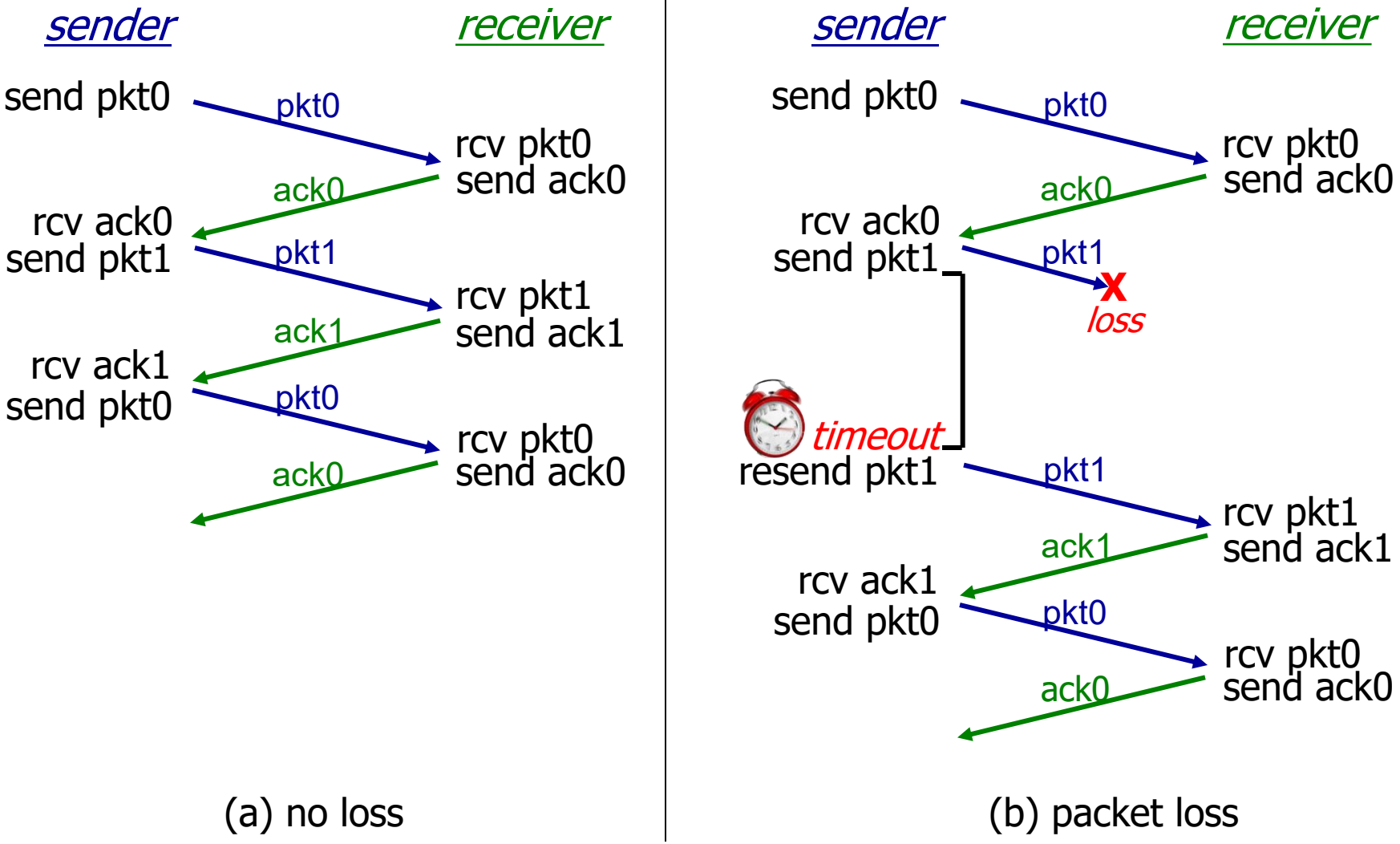
Question: What is a reasonable time?

TCP counts RTT – and makes that + some constant the timeout for the counter

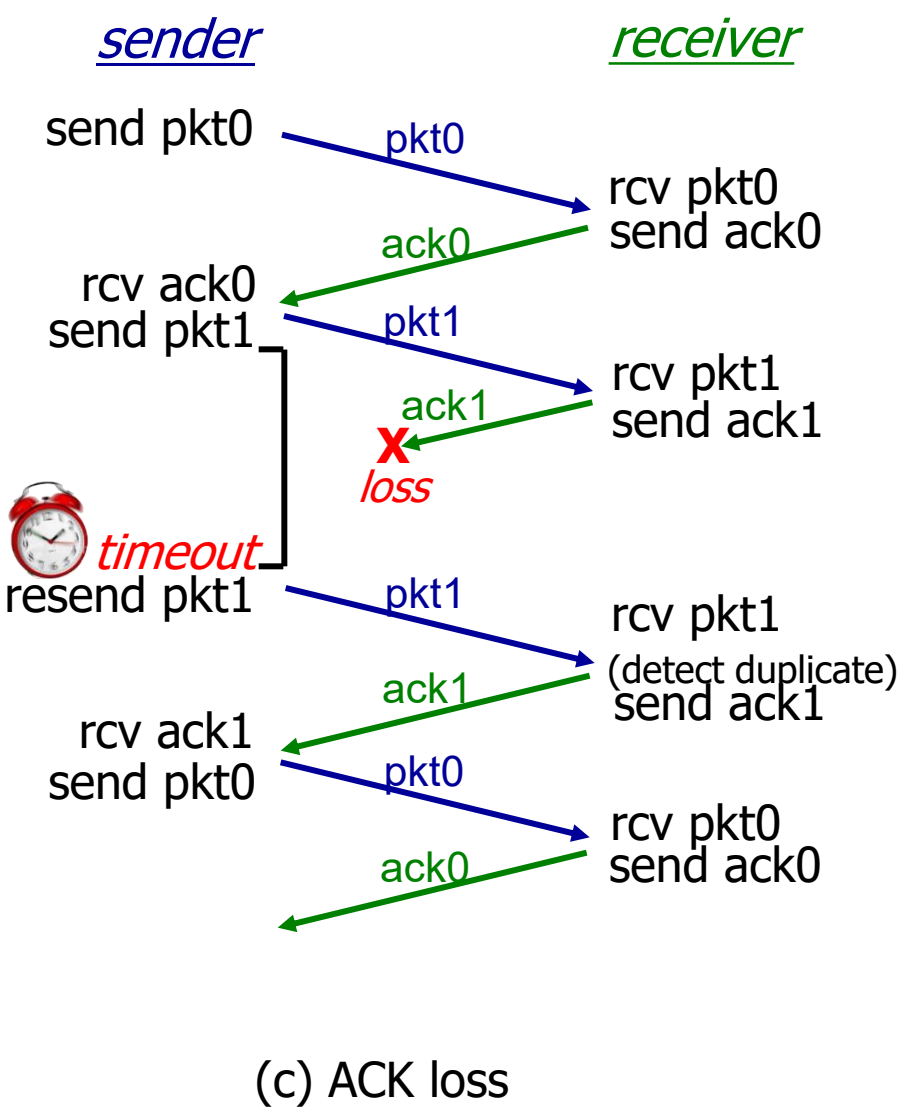
RDT-3.0: Sender Side



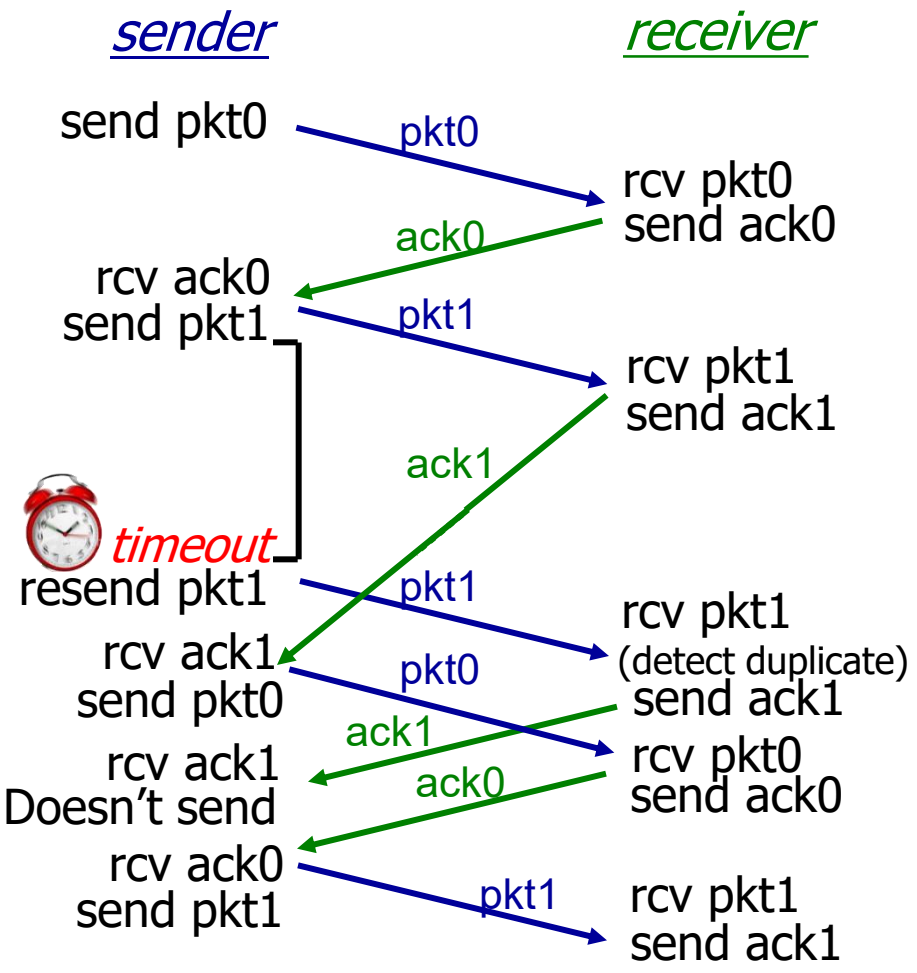
RDT-3.0: In Action Example



RDT-3.0: In Action Example



(c) ACK loss

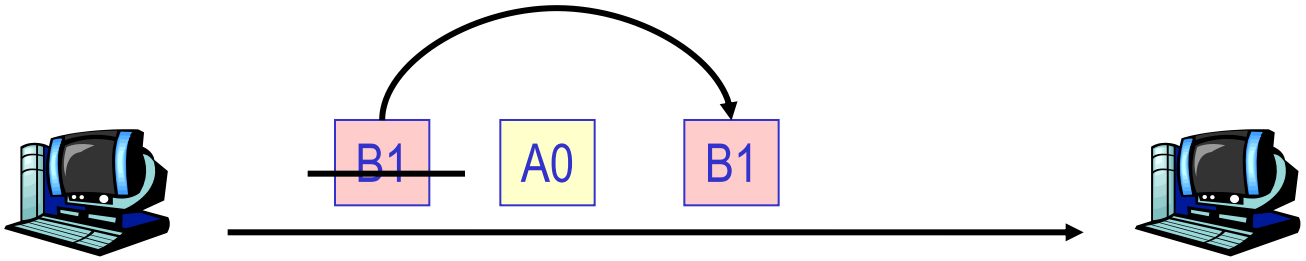


(d) premature timeout / delayed ACK

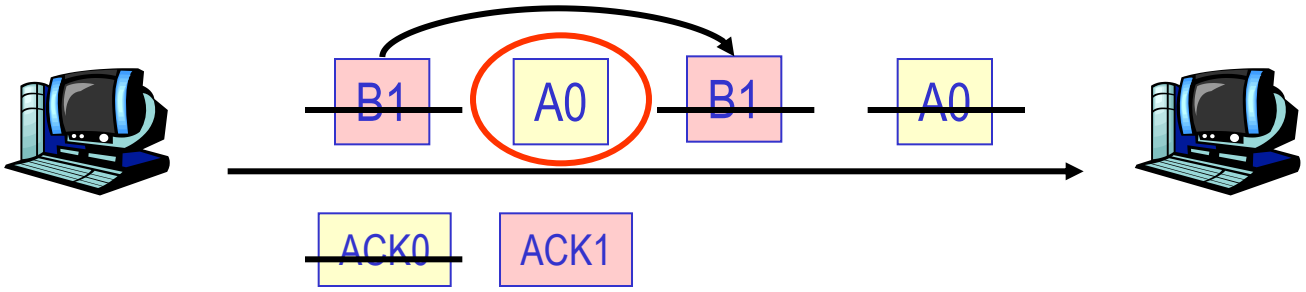
Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - Reorder

Does the Alternating Bit Protocol work correctly if the network connection between the sender and receiver can reorder messages?



Counter Example proving that the alternating bit protocol fails when reorder occur!



What can we do?

Use infinite number of sequence numbers

A question from an exam (5 points)

שאלה (5נקודות)

השאלה מתייחסת לפרוטוקול "הביט המתחלף". הנח שהודעות יכולות ללכת לאיבוד, ניתן לשכפל הודעות (ז"א, לשלוח אותה הודעה פעם נוספת) אבל הודעות לא מתקלקלות וגם הודעות מגיעות בסדר בו נשלחו. מה המספר המכסימלי של ההודעות שונות שיכולות להיות בתנועה מהשולח למקבל באותו זמן.

Consider the alternating bit protocol, and assume that messages can be duplicated or lost but messages are not corrupted and the medium does not reorder messages. What is the maximum number of different messages that can be in transit from the sender to the receiver at the same time? (Different messages are messages which have different data content.)

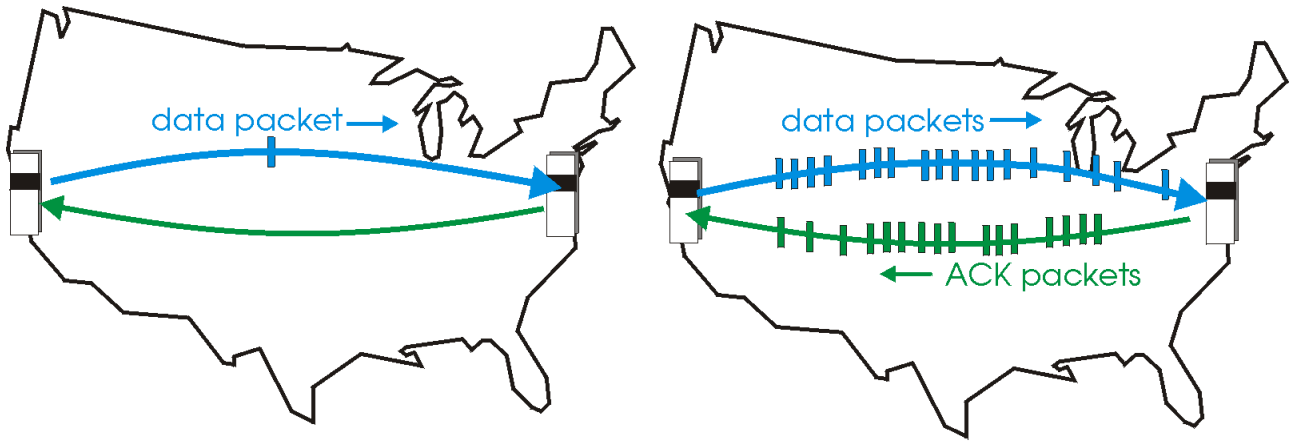
- No corrupted packets
- Duplications
- Loss
- FIFO (No reordering)

- C_0 is sent only after ack for B_1 arrived
- When ack for B_1 arrives (even if after multiple TO) the sender stops sending A_0
- Since B_1 is acked it has arrived to receiver
- Since no A_0 messages are sent after B_1 messages – all A_0 arrived to receiver (FIFO).
- So there are no A_0 messages on the line – leaving only 2 different messages possible.

Pipelined protocols

Pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts

- range of sequence numbers must be increased
- buffering at sender and/or receiver



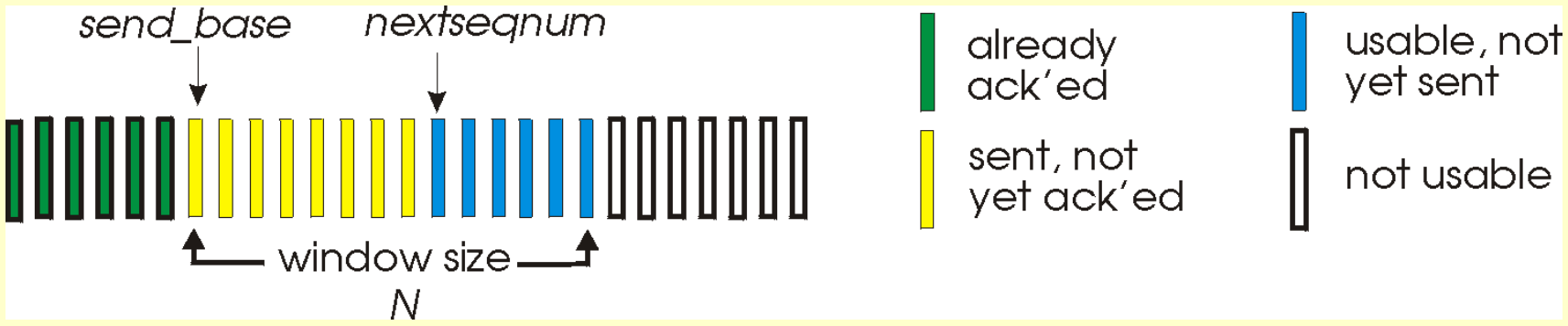
(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

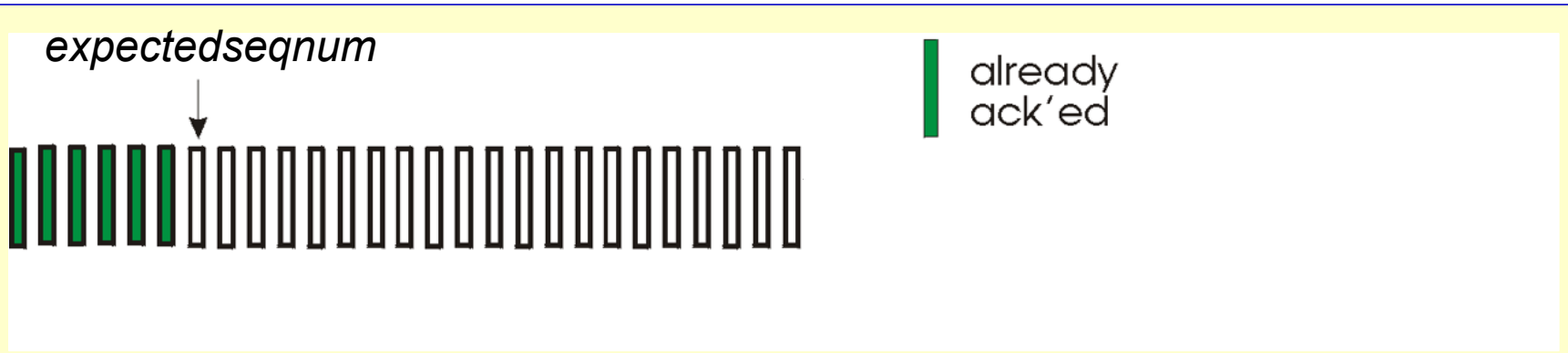
- Two generic forms of pipelined protocols: *go-Back-N, selective repeat*

The sliding-window based protocol

Sender



Receiver

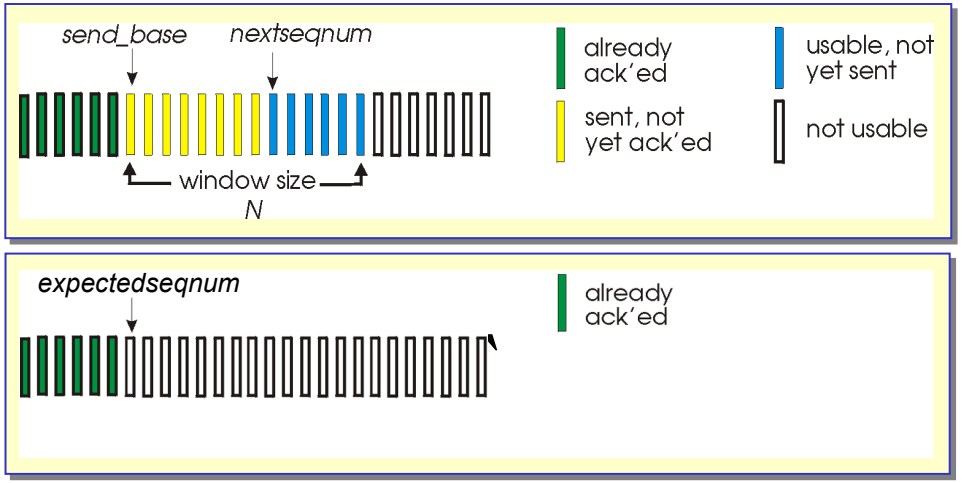


Go Back-N - Receiver

- Only **one** packet is acceptable at a time (the next expected).
- The receiver **never stores** out-of-order packets.
- Every ACK is cumulative.

- **When a packet arrives**
- Let the packet's sequence number be seqnum.
- If seqnum == expectedseqnum:
  - Accept the packet.
  - Deliver the data to the application.
  - Increment expectedseqnum.
  - Send an ACK for seqnum (the newly received in-order packet).
- Else (packet is out of order):
  - **Discard** the packet.
    Go-Back-N does not buffer out-of-order segments.
  - Re-send an ACK for the **last correctly received in-order packet**,
    which is expectedseqnum - 1.

- **Sending ACKs**
- ACKs are **cumulative**.
  Each ACK acknowledges *all* packets with sequence
  numbers < acknum.
- Therefore, the ACK number is always expectedseqnum - 1.



Go Back-N - Sender

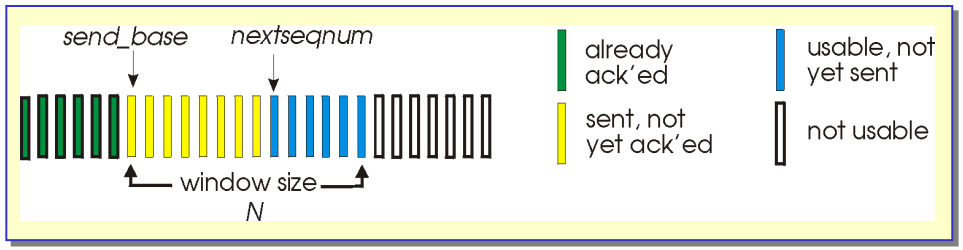
- Only **one** packet is acceptable at a time (the next expected).
- The receiver **never stores** out-of-order packets.
- Every ACK is cumulative.

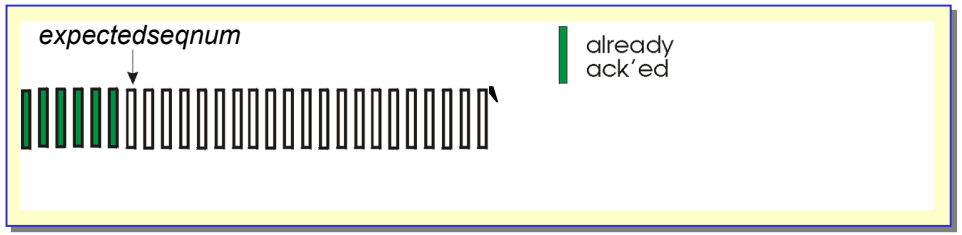
- **If (Exists unsent packs in the win):**
  - Construct packet with sequence number nextseqnum.
  - Send the packet.
  - If base == nextseqnum (window was previously empty), start the timer.
  - Increment nextseqnum.

  Otherwise:
  - The window is full; the sender must wait.

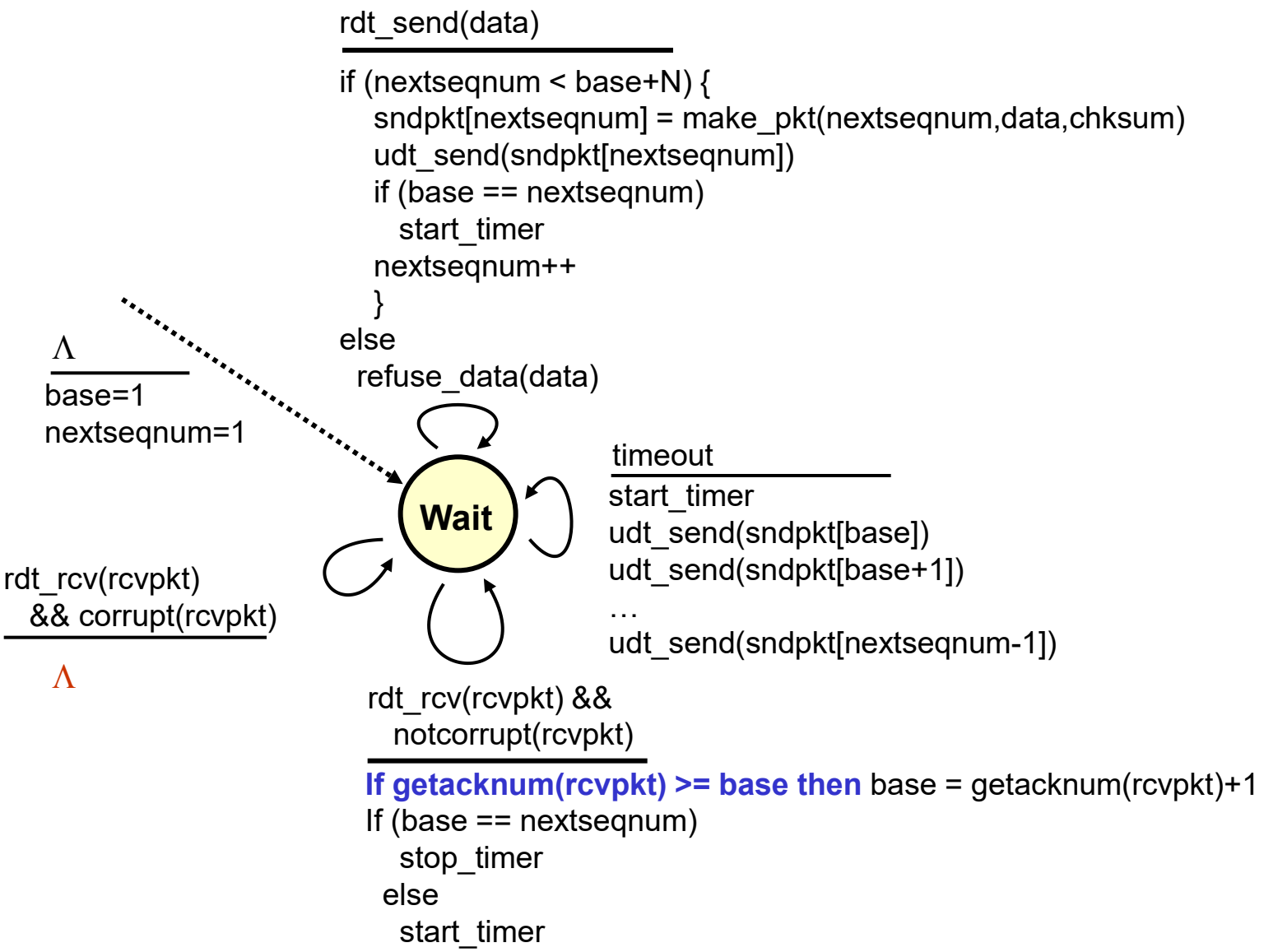
- **When an ACK is received**
- Let acknum be the acknowledged sequence number.
- If acknum >= base:
  - Set base = acknum + 1 (cumulative ACK).
  - If base == nextseqnum:
    stop the timer (no outstanding packets).
  - Otherwise:
    restart the timer for the oldest unacked packet (i.e., sequence base).

- **On Timer Expiration (Timeout Event)**
- The sender **retransmits all packets** in the window:
  - i.e., packets with sequence numbers from base to nextseqnum - 1.
- Restart the timer.

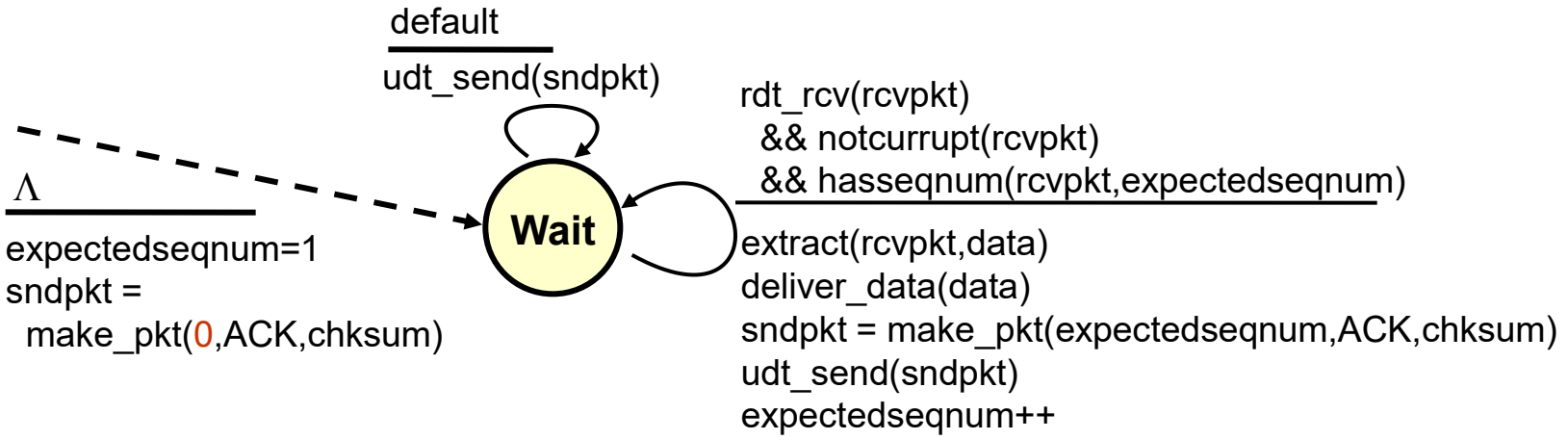




GBN: sender extended FSM



GBN: receiver extended FSM

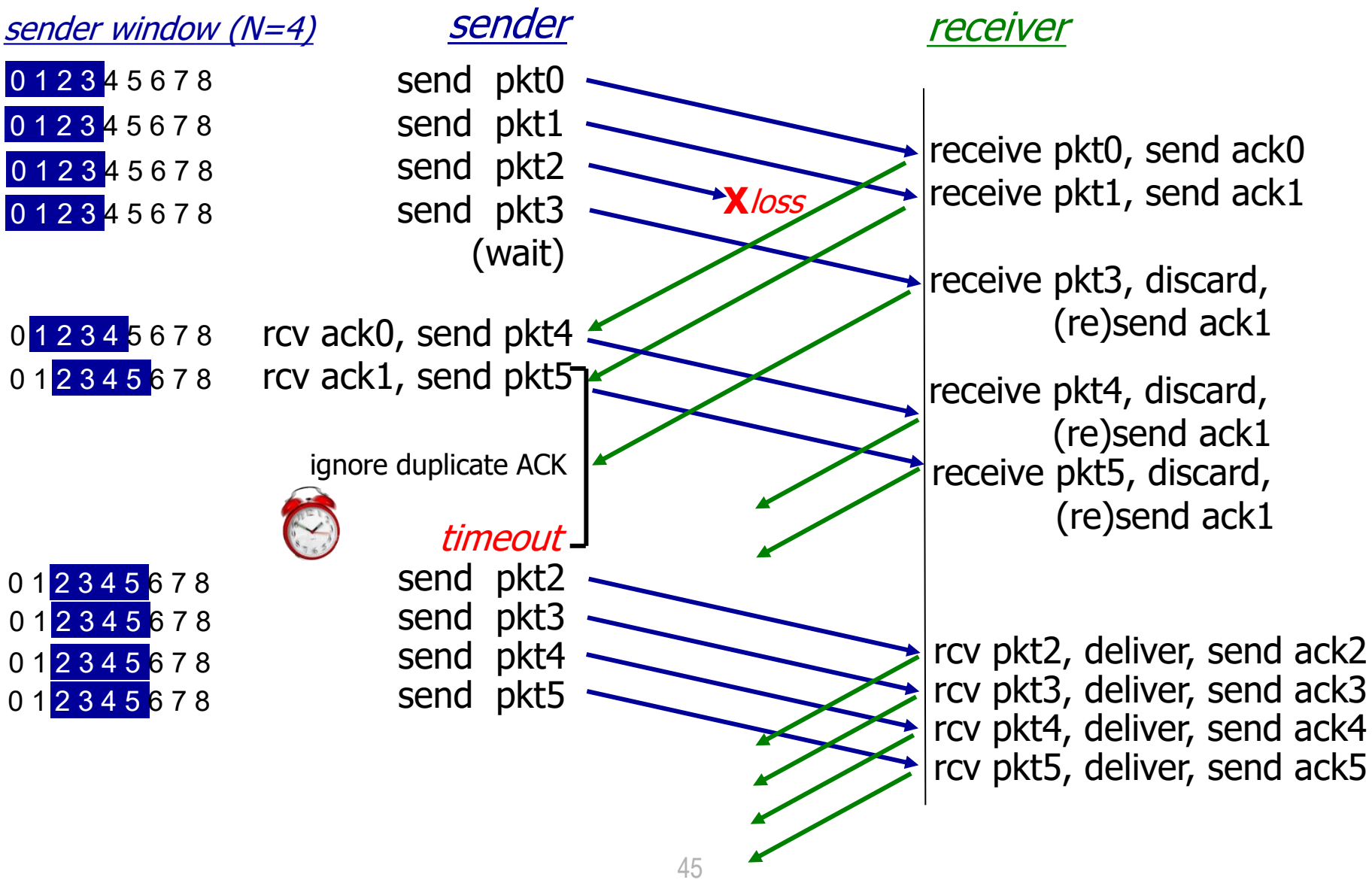


ACK-only: always send ACK for correctly-received pkt with highest *in-order* seq #

- may generate duplicate ACKs
- need only remember **`expectedseqnum`**

- Out-of-order pkt:
  - discard (don't buffer) -> no receiver buffering!
  - Re-ACK pkt with highest in-order seq #

GBN in action



Sliding Window - Selective Repeat

- receiver *individually* acknowledges all correctly received pkts
  - buffers packets, as needed, for eventual in-order delivery to upper layer

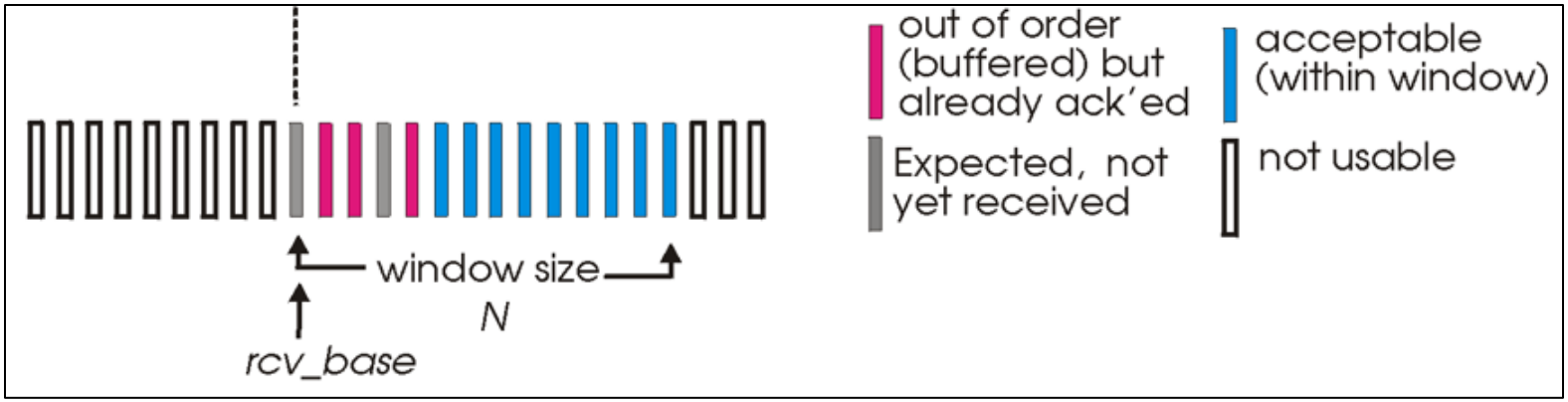
- sender only resends pkts for which ACK not received
  - sender timer for each unACKed packet
- sender window
  - N consecutive seq #'s
  - limits seq #s of sent, unACKed packets

Sliding window – Selective Repeat (Receiver)

- receiver **accepts and buffers out-of-order packets**.
  It delivers data to the application **only when all earlier packets have been delivered**.
- Uses per packet ACK

- **Receiver State**
  - **expectedseqnum** (or **rcv_base**): The lowest sequence number not yet delivered to the application.
  - **N**: window size.
  - **buffer[]**: storage for received but not-yet-delivered packets.
  - **received[]**: boolean flags indicating which packets in the window have arrived.

- **Packet arrives - Case A:**
  seqnum is inside the receiver window
- If the packet was **not previously received**:
  - Store it in the buffer.
  - Mark it as received.
- Send an ACK for seqnum
- If seqnum == rcv_base:
  - deliver full seq of received packets to app.
  - For every packet delivered, slide rcv_base forward..

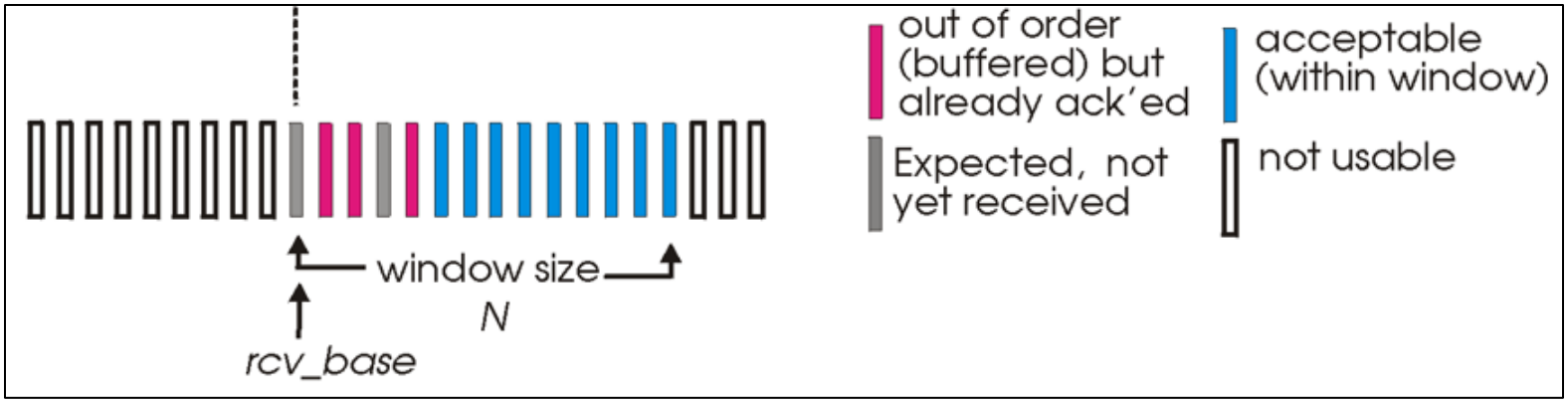


Sliding window – Selective Repeat (Receiver)

- receiver **accepts and buffers out-of-order packets**.
  It delivers data to the application **only when all earlier packets have been delivered**.
- Uses per packet ACK

- **Packet arrives - Case B:**
  seqnum is before the window
  - seqnum < rcv_base
    - This is a **duplicate** of a packet that has already been delivered.
    - Sender might need the ACK again.
  - Send an ACK for seqnum.

- **Packet arrives - Case C:**
  seqnum is above the window
  - seqnum ≥ rcv_base + N
    - Packet is outside the window and cannot be accepted.
    - Discard it.



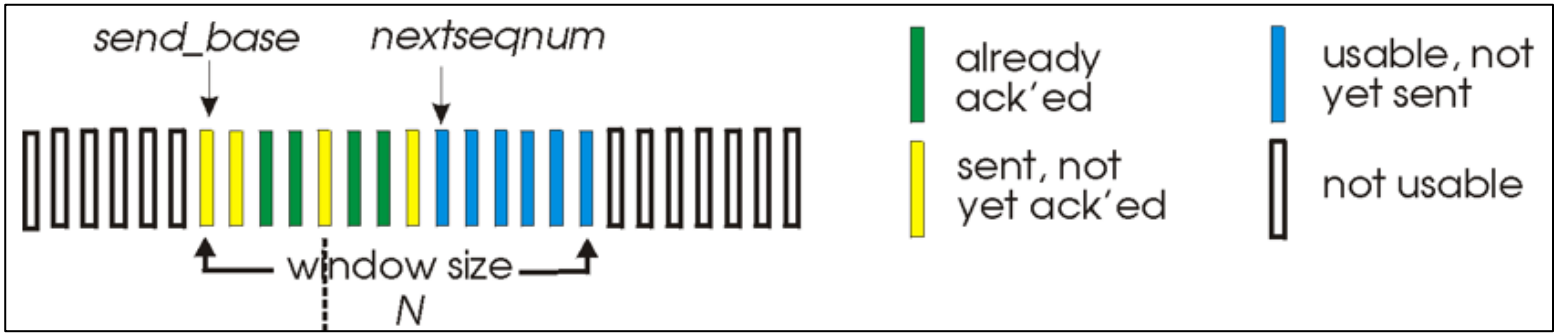
Sliding window – Selective Repeat (Sender)

- sender keeps a separate timer **per outstanding packet** and may retransmit each one individually. Its window allows transmission of up to **N unacknowledged packets** at a time.

- **Sender State**
  - **base** (or **snd_base**): The lowest sequence number of any unacknowledged packet.
  - **nextseqnum**: The next sequence number for sending.
  - **N**: Window size.
  - **packet[]**: A buffer containing sent-but-not-yet-acknowledged packets.
  - **timer[]**: Individual timers for each outstanding packet.

- **If the window is not full:**
  - Create a packet with sequence number nextseqnum.
  - Store it in the sender's buffer.
  - Send the packet.
  - Start a **per-packet timer** for nextseqnum.
  - Increment nextseqnum.

  Otherwise (window full):
  - The sender must wait; no transmission permitted until an ACK advances the window.



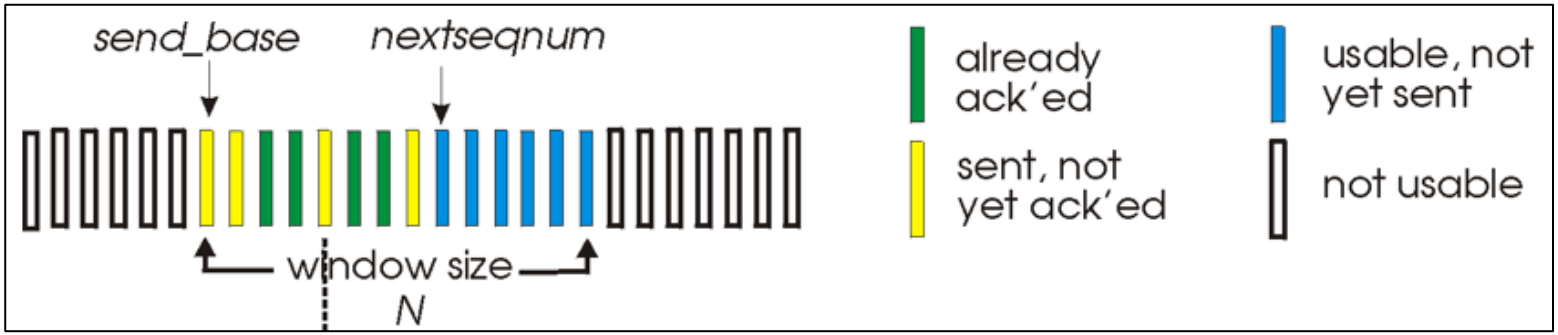
Sliding window – Selective Repeat (Sender)

- sender keeps a separate timer **per outstanding packet** and may retransmit each one individually. Its window allows transmission of up to **N unacknowledged packets** at a time.

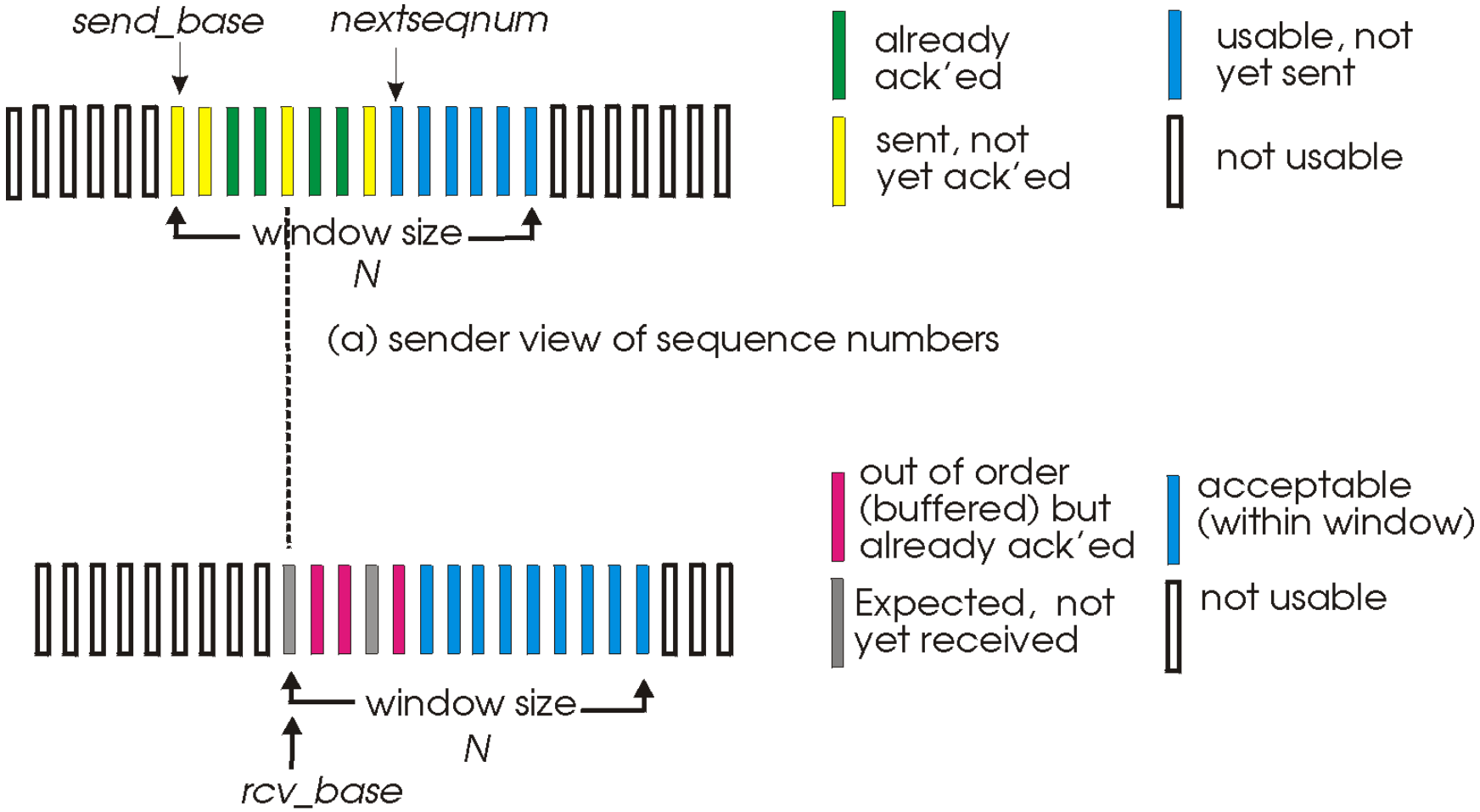
- If acknum is within the sender window:
  - Mark packet acknum as acknowledged.
  - Stop the timer for acknum.
  - If acknum == snd_base:
  - Move snd_base to the smallest sequence number not yet ACKed.
- ACKs outside the sender window are ignored.

- **On Timer Expiration (per packet)**

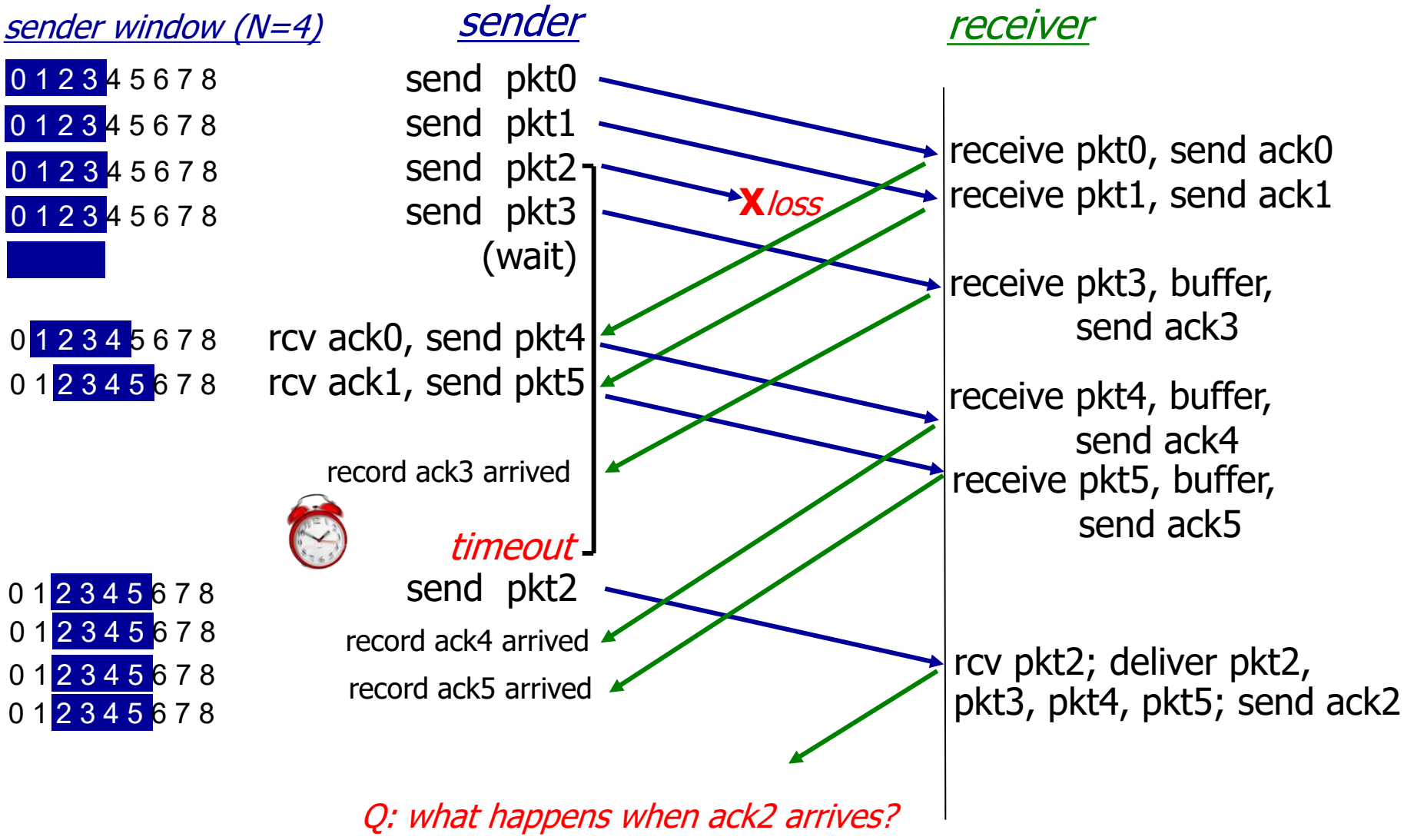
  For a timeout of packet i:
  - Retransmit packet i.
  - Restart timer for packet i.
- No other packets are retransmitted. This is the core difference from Go-Back-N.



Selective repeat: Sender, Receiver Windows



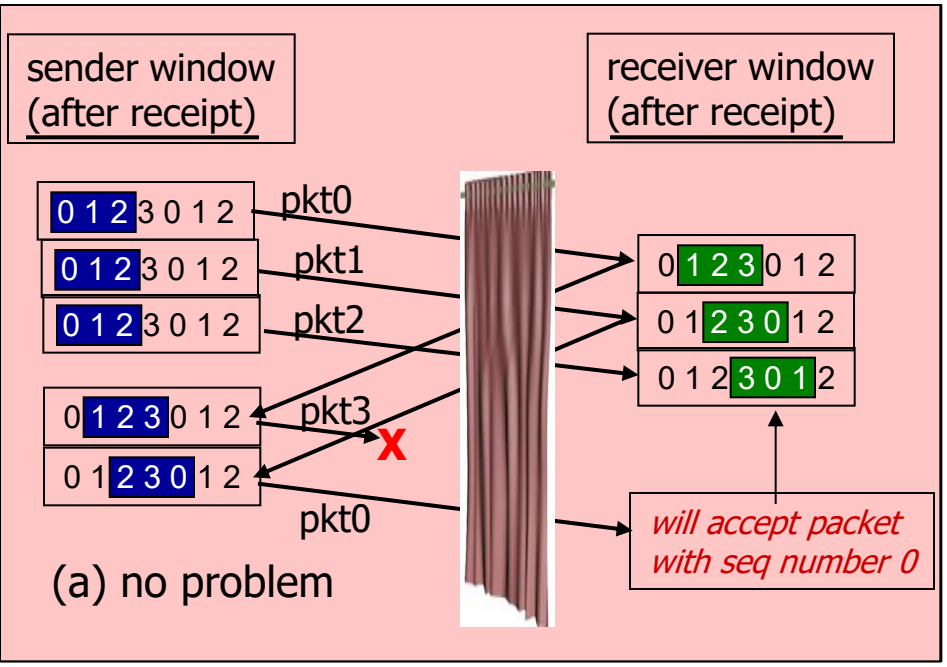
Selective Repeat Example

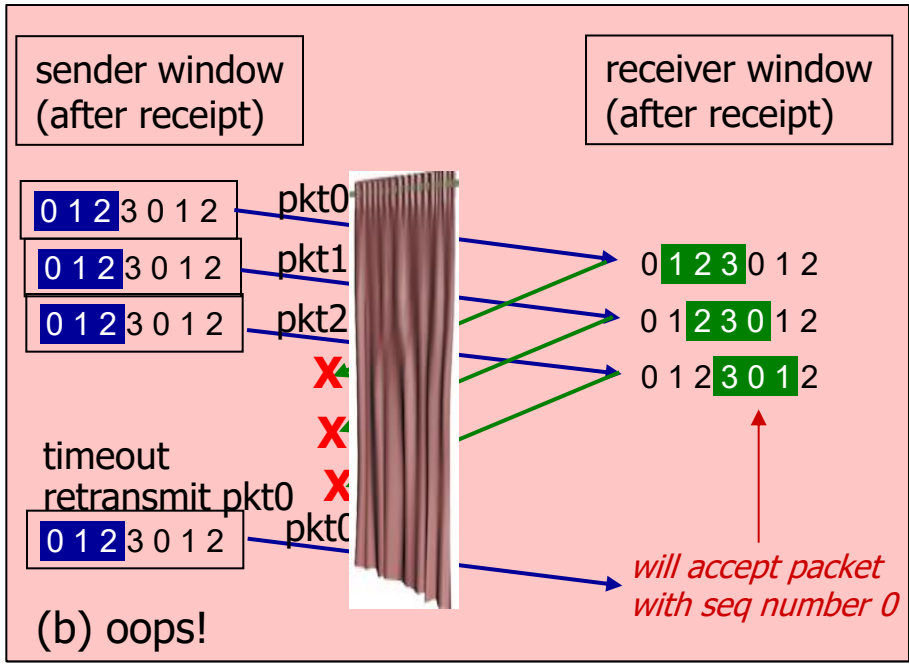


Selective Repeat – How many seq nums are needed?

example:

- seq #'s: 0, 1, 2, 3
- window size=3





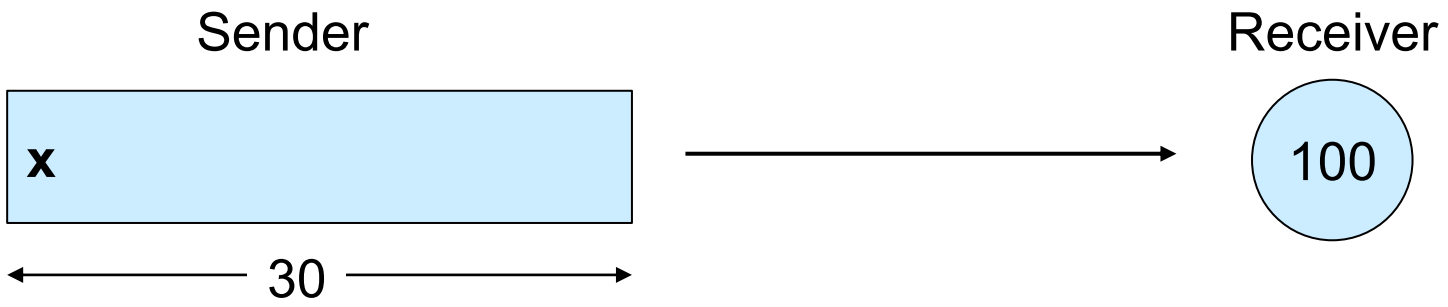
receiver can't see sender side.
receiver behavior identical in both cases!
something's (very) wrong!

A Question From Final Exam

השאלה מתייחסת לאלגוריתם Go-Back-N , כאשר גודל החלון של השולח הוא **30** ותחום ערכי המספרים הסידורים (sequence numbers) הוא מ-1 עד 2,048. הנח שבזמן מסוים שנסמן אותו ב- *t* המקבל מחכה לקבל הודעה עם מספר סידורי **100** (ז״א המקבל כבר קיבל את כל ההודעות עד מספר סידורי 99 כולל).

א. מהם כל הערכים הסידורים האפשריים שיכולים להופיע <u>במקום הראשון</u> בחלון של השולח בזמן *t* ? נמק את תשובתך.

ב. מהם כל הערכים הסידוריים האפשריים בהודעות ה- ACK הנמצאות בתנועה מהמקבל אל השולח בזמן *t* ? נמק את תשובתך. הנח שהודעות ה- ACK מגיעות לשולח <u>בסדר בו נשלחו</u> על ידי המקבל.

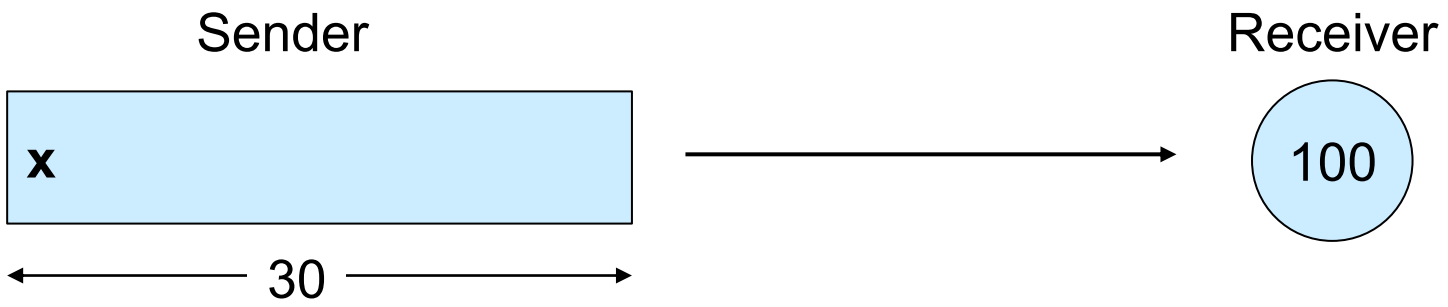


A Question From Final Exam

Consider the Go_Back-N protocol with a sender widow size of **30** and sequence number range from 1 to 2,048. Suppose that a time ***t***, the next in-order packet that the receiver is expecting has a sequence number **100**. (That is the receiver has already accepted all the messages up to 99.)

1. What are the possible sequence numbers that can appear at the beginning of the sender's window at time *t* ? Justify your answer.

2. What are all the possible values of the ACK field in all possible messages currently propagating back from the receiver to the sender at time *t* ? Justify your answer. (Assume that the medium does not reorder messages.)



A Question From Final Exam

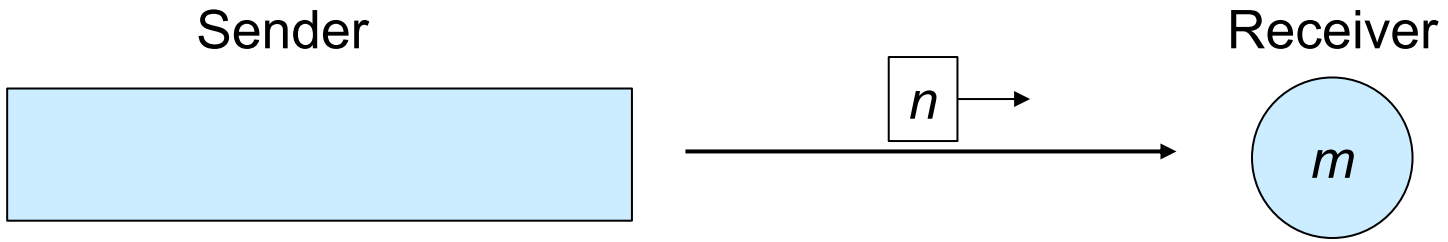
הנח שאנחנו משנים את אלגוריתם **Go-Back-N** של **המקבל** באופן הבא:
נניח שהמקבל מחכה לקבל חבילה עם מספר סידורי ***m***. (ז"א, המקבל כבר קיבל את כל החבילות עם מספר סידורי הקטן מ- m באופן תקין ולפי הסדר). בשלב זה, כאשר מגיעה חבילה תקינה עם מספר סידורי ***n*** , אם $n=m$ המקבל מתנהג כמו באלגוריתם המקורי ושולח ACK עבור חבילה n . אחרת:

עבור כל אחד משלושת המקרים, החליטו כיצד השינוי משפיע על נכונות האלגוריתם. נמק את תשובתך.

מקרה א: אם $n > m$, המקבל זורק את החבילה **וגם** שולח שוב ACK עבור חבילה m-1 ;
אם $n < m$ המקבל זורק את החבילה ולא שולח ACK.

מקרה ב: אם $n > m$, המקבל זורק את החבילה ולא שולח ACK ;
אם $n < m$ המקבל זורק את החבילה **וגם** שולח שוב ACK עבור חבילה m-1 .

מקרה ג: אם $n > m$, המקבל זורק את החבילה ולא שולח ACK ;
אם $n < m$ המקבל זורק את החבילה ולא שולח ACK.



A Question From Final Exam

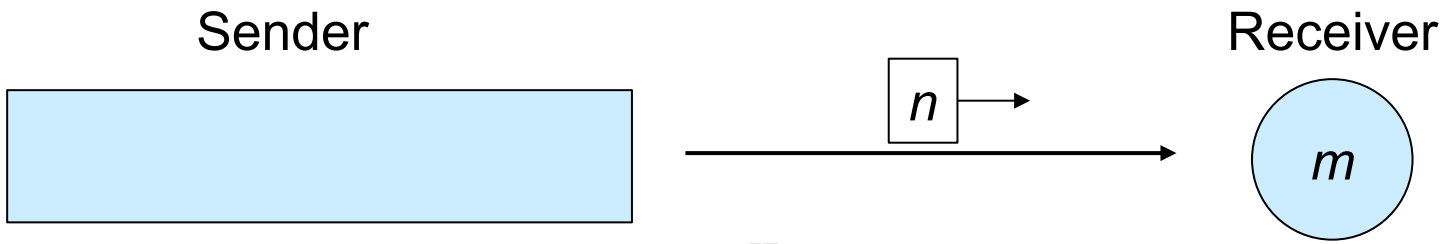
Assume that we change the **Go-Back-N** algorithm of the **receiver**, as follows: Let's assume that the receiver is now waiting to receive packet with sequence number ***m***. (Than is, the receiver has already received correctly and in-order all the packets with sequence numbers smaller than *m*.) At that point, when a packet with sequence number ***n*** is received correctly then, if $n=m$ the receiver behave as in the original algorithm and sends an ACK for packet *n*. Otherwise:

For each one of the three cases decided how the change affects the correctness of the algorithm. Justify your answer.

1. if $n > m$, the receiver discards the packet **and** resends an ACK for packet m-1; if $n < m$, the receiver discards the packet **and** does not send an ACK.

2. if $n > m$, the receiver discards the packet **and** does not send an ACK; if $n < m$, the receiver discards the packet **and** resends an ACK for packet m-1.

3. if $n > m$, the receiver discards the packet **and** does not send an ACK; if $n < m$, the receiver discards the packet **and** does not send an ACK.



Go-Back-N ♦ Selective Repeat ♦ TCP

- ACKs are cumulative ?
  - Go-back-N: Yes
  - Selective Repeat: No
  - TCP: Yes

- When one segment is lost, retransmit one segment ?
  - Go-back-N: No
  - Selective Repeat: Yes
  - TCP: Yes

- Must buffer, received but out of order segments ?
  - Go-back-N: No
  - Selective Repeat: Yes
  - TCP: No, but

- One timer ?
  - Go-back-N: Yes
  - Selective Repeat: No
  - TCP: Yes

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ • What can go wrong
    - ✓ • Error control
    - ✓ • Duplication
    - ✓ • Loss
    - ✓ • Reorder